Caderno Âmbar por dentro

Manual de estudo de Python, arquitetura e funcionamento do projeto

Edição revisada em 4 de novembro de 2026, com a organização das pastas atualizada. Preparado para Victor Monteiro, com base nos arquivos locais desta revisão.

1 Como estudar este documento

Este manual explica como o Caderno Âmbar foi montado, onde cada responsabilidade está localizada e como acompanhar uma ação do usuário até o banco e a fonte de dados. Ele serve para estudar, modificar e demonstrar o projeto. Não é uma declaração de que o aplicativo está homologado para operação jurídica em produção.

A primeira leitura deve seguir os capítulos 2 a 8. Eles constroem a visão geral e ensinam o vocabulário Python usado no código. Depois, estude os arquivos do servidor nos capítulos 9 a 15, a interface nos capítulos 16 a 19 e os testes e exercícios finais. O catálogo complementar mapa-do-codigo.md lista os símbolos encontrados no código e suas responsabilidades, com linhas de referência da revisão.

Abra o projeto no VS Code ao lado do manual. Não tente decorar nomes de métodos. Para cada fluxo, procure responder: quem chama, o que entra, o que é validado, o que é gravado, o que sai e o que acontece se falhar. Saber responder a essas perguntas mostra mais domínio do que repetir nomes de bibliotecas.

Os exemplos de código deste manual são recortes ou simplificações didáticas. Quando houver diferença, o arquivo local é a implementação de referência. Não copie exemplos para um banco real sem compreender o efeito. Os exemplos não contêm credenciais reais.

2 O problema que o aplicativo resolve

Um escritório acompanha vários processos e precisa saber quando uma informação mudou, quem leu essa mudança e qual trabalho foi feito a partir dela. Consultar a fonte é apenas uma parte do problema. Também é necessário preservar a referência anterior, distribuir responsabilidades e registrar a conclusão.

O Caderno Âmbar organiza esse ciclo: cadastrar processo, consultar, comparar, conferir e criar uma providência. Uma versão representa um conteúdo recebido da fonte. Uma conferência registra a leitura humana de uma versão. Uma providência representa trabalho que alguém deve executar e outra pessoa pode revisar. Esses três conceitos têm registros e estados próprios.

Exemplo fictício: a primeira consulta informa distribuição. Outra consulta encontra uma audiência. O aplicativo guarda uma versão nova, mostra a diferença e registra uma pendência de leitura. A pessoa confere a informação e cria a providência “Preparar documentos para análise”, atribui responsável e revisor e informa uma data interna. A revisão da tarefa não substitui a leitura da fonte e não é uma prática de ato judicial.

O produto atual é focado em acompanhamento e rotina. Não oferece acesso integral aos autos, peticionamento, financeiro, validação jurídica ou contagem automática de prazos. O módulo de treino utiliza dados fictícios. O transporte DataJud está preparado no código, mas o uso real depende de configuração, condições de acesso e validação com uma carteira autorizada.

3 Arquitetura em uma página

Há quatro camadas principais. A interface é HTML, CSS e JavaScript no navegador. A API é a porta HTTP do servidor, construída com FastAPI. Os serviços Python aplicam regras e acessam persistência. As fontes externas ou fictícias fornecem a informação consultada. O SQLite guarda contas, processos, versões e tarefas em um arquivo local.

O caminho normal é navegador -> FastAPI -> sessão e autorização -> serviço -> SQLite ou fonte -> serviço -> JSON -> interface. A tela não acessa o SQLite diretamente. A chave de uma fonte pertence ao servidor; não deve aparecer em JavaScript enviado ao visitante.

O projeto contém duas gerações de servidor. backend/servidor.py é o laboratório legado na porta 8092. backend/aplicacao.py é a entrada recomendada da aplicação de equipes na porta 8093. Manter os dois arquivos ajuda a estudar a evolução, mas eles não têm os mesmos recursos. Abrir a tela atual sobre o servidor antigo pode provocar erros de contrato.

A parte persistente usa uma cadeia de herança: Caderno -> Equipes -> Escritorio -> Piloto. Caderno contém a base de contas e armazenamento. Equipes acrescenta escritórios, versões, importação e tarefas. Escritorio acrescenta providências e processamento de filas. Piloto acrescenta confirmação de e-mail, carteira paginada, resumo e exportação administrativa. A API instancia Piloto; ao chamar um método, Python procura primeiro nessa classe e depois nas bases.

Essa evolução evita reescrever todo o sistema, mas também tem um custo: para entender um método herdado, pode ser necessário abrir mais de um arquivo. Uma refatoração futura pode separar repositórios, autenticação e serviços de domínio por composição. Não é necessário trocar a arquitetura inteira para corrigir uma função pequena.

4 Pastas e arquivos da raiz

backend contém o servidor Python, regras, integrações e testes Python. frontend contém as páginas e recursos entregues ao navegador. tests contém os testes JavaScript em DOM simulado. ferramentas reúne utilitários de documentação. documentacao reúne os guias, o catálogo e as versões de leitura do manual. output contém banco, mensagens locais, backups e resultados de execução; não deve ser versionada como código público.

requirements.txt fixa versões das dependências Python usadas no ambiente. FastAPI constrói a aplicação HTTP; Uvicorn executa o servidor ASGI; Requests faz chamadas HTTP de saída; Beautiful Soup extrai HTML; Playwright automatiza o navegador no roteiro de estudo. Algumas dependências são transitivas, como certifi e urllib3, usadas pelo transporte HTTP. Ter Playwright instalado não significa que ele participa de todas as consultas.

package.json descreve os comandos dos testes de interface e a dependência jsdom. package-lock.json registra a resolução dessas dependências. node_modules contém os pacotes instalados e não é fonte autoral do projeto. Não é necessário estudar cada arquivo de uma biblioteca para entender o aplicativo, mas é necessário saber o contrato que ela fornece.

INICIAR.cmd prepara um ambiente .venv, instala requirements.txt e inicia a aplicação. .venv isola bibliotecas Python do projeto; ela não contém o código de negócio. O lançador exige Python disponível e acesso aos pacotes quando a instalação ainda não foi feita. Não é um instalador de produção.

.env.example documenta variáveis esperadas. O programa atual lê os valores do ambiente do processo com os.environ; ele não carrega esse arquivo automaticamente. Copiar .env.example para .env, sem configurar um carregador, não ativa os valores. A configuração pode ser passada pelo terminal ou pelo serviço de hospedagem.

.gitignore impede que arquivos sensíveis ou gerados sejam incluídos normalmente no Git. Ele não remove um segredo que já tenha sido publicado. .dockerignore reduz o contexto enviado ao Docker. Dockerfile descreve uma execução Linux com usuário sem privilégios e banco em /data. A receita ainda precisa ser construída e verificada no ambiente de destino.

vercel.json pertence à publicação estática existente. Servir frontend pela Vercel não coloca automaticamente a aplicação Python persistente no ar. Para a versão completa, é necessário hospedar a API e o armazenamento de forma compatível e revisar origem, rotas e HTTPS.

README.md apresenta o projeto. primeiro-guia-de-estudo.md documenta o primeiro laboratório. guia-de-equipes.md explica a evolução de equipes. guia-de-providencias.md detalha providências e filas. revisao-da-interface.md registra ajustes de experiência. fluxo-das-telas.md explica a divisão entre apresentação, treino, acesso e uso. testes-e-validacao.md reúne evidências por etapa. estado-atual-do-projeto.md é o estado atual de implementação e pendências. Este manual integra a explicação para estudo; os registros anteriores permanecem como histórico.

5 Python para quem vem de Java

Python usa indentação para delimitar blocos. Uma definição começa com def e dois pontos; o corpo fica recuado. Diferentemente de Java, uma variável não precisa declarar o tipo antes do nome. Isso não significa ausência de tipos: cada valor tem um tipo em tempo de execução. As anotações auxiliam leitura e ferramentas, mas não substituem validação dos dados de uma requisição.

```
def saudacao(nome: str) -> str:
    return 'Olá, ' + nome
```

def cria a função; nome é um parâmetro; str indica intenção de receber texto; a seta indica retorno textual; return entrega um resultado ao chamador. Uma chamada saudacao('Victor') devolve um novo texto. Se uma função apenas exibe algo com print, isso não equivale a devolver um resultado para a API.

Listas representam sequências, como as movimentações. Dicionários associam chaves a valores, como um processo com numero e tribunal. Tuplas são sequências imutáveis frequentemente usadas como parâmetros SQL. Conjuntos ajudam a comparar ou eliminar duplicações. None representa ausência de valor. True e False são booleanos, com inicial maiúscula em Python.

```
caso = {'numero': 'DEMO-1', 'tribunal': 'tjpe'}
```

```
numero = caso.get('numero', '')
movimentos = [{'codigo': 26}, {'codigo': 123}]
codigos = {m['codigo'] for m in movimentos}
```

caso['numero'] exige que a chave exista. caso.get('numero', '') devolve uma alternativa quando ela não existe. A compreensão de conjunto percorre a lista e extrai códigos únicos. No projeto, compreensões aparecem em validação de schema, transformação de resultados e comparação de movimentos. Para estudar, reescreva uma compreensão como um for com append ou add e compare os resultados.

Operadores and e or podem devolver os próprios operandos, não apenas booleanos. Por isso cliente or Demonstracao() escolhe cliente quando ele existe; caso contrário, constrói uma fonte demonstrativa. Já is None verifica ausência de maneira explícita. Não confunda texto vazio, número zero e None quando o campo de negócio distingue essas situações.

f-strings interpolam valores: f'casos/{id}/consultar'. Use-as para textos e caminhos controlados. Em SQL, valores recebidos devem ser parâmetros com ?, não interpolação. Um identificador de tabela dinâmico exige lista interna controlada, porque parâmetros SQL normalmente representam valores, não nomes de tabelas.

6 Classes e herança no projeto

Uma classe agrupa estado e comportamento. self é a referência à instância atual, comparável ao this de Java. __init__ inicializa o objeto. super() chama comportamento da classe base. Métodos com @staticmethod não recebem self porque não dependem do estado da instância.

```
class Piloto(Escritorio):
    def __init__(self, arquivo):
        super().__init__(arquivo)
        self.migrar_carteira()
```

Nesse recorte, Piloto recebe o caminho do banco. O construtor da base prepara as estruturas anteriores; depois a nova classe aplica a própria migração. Não copie uma criação de objeto para dentro de cada requisição: isso repetiria verificações de schema desnecessariamente. criar_app cria o serviço e o guarda no estado da aplicação.

Quando Piloto sobrescreve convidar, ele exige e-mail confirmado e chama super().convidar para aproveitar a regra já existente de administrador, token e destino. Um teste apenas de Equipes não comprova essa nova exigência: é necessário um teste de Piloto ou da API que o utiliza.

Fonte, em fontes.py, usa Protocol. Ele descreve a forma esperada de uma fonte: disponibilizar buscar(tribunal, numero). Demonstracao e DataJudValidado podem atender a esse contrato sem herdar uma mesma implementação concreta. Isso permite injetar uma fonte de teste, simulando falha ou demora sem depender da internet.

7 Erros e recursos com fechamento garantido

FalhaAPI carrega um status HTTP e uma mensagem compreensível. O serviço pode lançar FalhaAPI(403, 'Acesso recusado.'); a camada HTTP converte a falha em JSON e código de resposta. raise interrompe o

caminho normal. try e except tratam erros esperados. finally executa a limpeza mesmo quando há erro.

O bloco with representa um gerenciador de contexto. Em conexao, o projeto abre sqlite3, configura o tipo das linhas e garante fechamento ao sair. O with db confirma a transação quando tudo dá certo e desfaz alterações quando ocorre uma exceção. O decorador @contextmanager transforma uma função com yield em um contexto utilizável por with.

```
with self.conexao() as db:
    db.execute('BEGIN IMMEDIATE')
    registro = db.execute('SELECT id FROM casos WHERE id=?', (id,)).fetchone()
```

O ? recebe o parâmetro id separadamente. (id,) é uma tupla com um elemento; a vírgula faz diferença. fetchone retorna uma linha ou None. fetchall devolve as linhas encontradas. dict(linha) transforma uma sqlite3.Row em dicionário para serialização e uso no serviço.

Não use except Exception para transformar qualquer erro em sucesso. O projeto registra erros inesperados e preserva a referência anterior. Nas fronteiras, uma mensagem pública deve evitar revelar senha, token, conexão ou caminho interno. O log técnico deve permitir investigação sem expor conteúdo sensível desnecessário.

8 HTTP e comunicação entre interface e servidor

HTTP é o protocolo entre navegador e API. GET consulta informação; POST envia uma ação ou dados. O corpo JSON é uma representação de objetos, listas, textos, números e booleanos. JSON não é uma conexão com o banco e não executa Python no navegador.

No frontend, fetch chama /api/carteira com parâmetros como org, pagina, busca e filtro. No servidor, FastAPI identifica a rota, converte parâmetros tipados, lê o cookie e chama o serviço. A resposta vira JSON. O JavaScript lê a resposta, verifica response.ok e atualiza o DOM.

Códigos usados no projeto: 200 indica resposta bem-sucedida; 400 indica dados inadequados; 401 exige autenticação; 403 recusa uma ação ou escritório; 404 indica recurso inexistente ou indisponível naquele escopo; 409 indica conflito, duplicação ou revisão desatualizada; 413 recusa corpo grande; 422 pode vir da validação tipada do FastAPI; 429 aplica limite temporário; 502 descreve falha do serviço externo; 503 indica configuração ou serviço indisponível.

O navegador recebe um cookie de sessão HttpOnly. Ele o envia em chamadas da mesma origem, mas o JavaScript não deve ler seu conteúdo. O SQLite guarda o hash do token. Autenticar é descobrir quem é a pessoa; autorizar é verificar se aquela pessoa pode trabalhar naquele escritório ou alterar aquela providência. Esconder um botão não substitui a autorização no servidor.

9 aplicacao.py e a entrada recomendada

criar_app é uma fábrica: recebe um banco opcional e a opção de executar ciclos, cria Piloto, define rotas, middleware e recursos estáticos e devolve FastAPI. Isso permite criar uma aplicação de teste com banco temporário e sem tarefas de fundo. No uso local, o bloco __main__ chama Uvicorn.

Em produção, a fábrica exige origem HTTPS e configuração de banco e SMTP. Essa verificação evita iniciar uma instalação claramente incompleta, mas não comprova que o domínio, o certificado, o banco e o provedor funcionam. AMBAR_WORKER permite desativar os ciclos em processos HTTP extras.

lifespan cria tarefas assíncronas para consultas, e-mails e backups. Ao encerrar, cancela e aguarda as tarefas. agendador repete uma função em intervalos, registra falhas e usa run_in_threadpool para executar trabalho síncrono sem bloquear diretamente o laço assíncrono. asyncio.sleep cede execução; não é uma espera ocupada.

controles é o middleware aplicado às requisições. Ele verifica origem, tamanho do corpo e características de requisições entre sites. Também acrescenta identificador de requisição, política de conteúdo, proteção contra interpretação incorreta de tipo e regras de cache. O TrustedHostMiddleware restringe hosts aceitos. Essas proteções não constituem uma auditoria completa de segurança.

limitar grava contadores temporários no SQLite, dentro de uma transação. É usado em autenticação, recuperação, confirmação e consultas. Esse mecanismo local atende ao desenho atual de uma instância, mas pode exigir outra estratégia em uma implantação distribuída. O IP considerado vem da conexão; confiar em cabeçalhos de proxy exige configuração deliberada.

dados verifica application/json e exige objeto JSON. endpoint encaminha rotas legadas para dispatch em uma thread. dispatch organiza autenticação, contexto, processos, equipe, rotina e exportações. A rota nova /api/carteira tem parâmetros explícitos no FastAPI. As demais rotas ainda estão agrupadas em um roteador genérico; não existe documentação OpenAPI detalhada de cada uma.

saude verifica uma conexão básica com o banco. Retornar status ok não comprova a saúde do SMTP ou a disponibilidade do tribunal. runtime serve a configuração JavaScript do laboratório. StaticFiles entrega as páginas de frontend. Em desenvolvimento, HTML, CSS e JavaScript recebem no-store para reduzir problemas de cache durante revisão.

10 acompanhamento.py e a base persistente

FalhaAPI é a exceção de domínio usada pela API. Caderno.__init__ prepara o caminho do banco e cria a estrutura original: usuarios, sessoes, casos e consultas. Essa base pessoal nasceu antes dos escritórios. Seu listar e seu revisar representam o contrato antigo; na versão atual, parte dessas operações é sobrescrita por classes mais recentes.

hash_senha usa scrypt com salt aleatório. Salt é um valor associado à senha para impedir que duas senhas iguais produzam automaticamente o mesmo resultado armazenado. O banco não precisa guardar a senha original para conferir login. hmac.compare_digest compara os resultados de forma apropriada para esse contexto. Hash não é criptografia reversível.

autenticar valida e-mail e tamanho da senha. No cadastro, insere usuário com hash. No login, procura a conta e compara a senha calculada. Ao autenticar, gera token aleatório, armazena apenas SHA-256 do token e registra expiração. O token original é devolvido à camada HTTP para virar cookie; não deve aparecer em logs.

usuario resolve um token de sessão válido em um ID de usuário. sair elimina essa sessão. Alterar a senha pelo fluxo de recuperação elimina as sessões existentes, obrigando novo login. A confirmação de e-mail é uma regra posterior, em Piloto, e não deve ser confundida com o sucesso do login.

DataJud.buscar faz a requisição ao endpoint oficial correspondente ao tribunal. Envia Authorization no servidor, consulta o número exato, pagina os resultados, preserva os graus retornados e organiza movimentações. Há limites de tempo, volume e quantidade de páginas. Atingir o limite gera falha em vez de gravar uma resposta truncada como completa.

O transporte não abre navegador e não utiliza Beautiful Soup, porque recebe JSON. Requests é a biblioteca HTTP usada nesse caminho. O código de extração Beautiful Soup pertence ao laboratório HTML de consulta.py. Essa distinção é importante para explicar corretamente o projeto na entrevista.

A conexão do DataJud é direta por padrão e mantém a validação TLS. Para herdar os proxies configurados no ambiente, use AMBAR_DATAJUD_USAR_PROXY=1. O transporte diferencia limite e sobrecarga, respeita Retry-After e preserva a referência anterior. Consulta páginas de dez registros com projeção de campos e search_after ordenado por @timestamp. Respostas parciais são recusadas. A tabela fontes_estado guarda a espera e a reserva da fonte, também respeitadas pela consulta manual. Em 04/11, o fluxo autenticado recebeu 35 movimentos do TJPE e gravou uma versão real, preservada após novo login. Veja diagnostico-e-correcao-datajud.md para evidências e limites.

Atualização em segundo plano

O botão Solicitar atualização usa POST /api/casos/{id}/agendar. HTTP 202 significa que o pedido foi salvo na fila; não significa que o CNJ já respondeu. Piloto verifica e-mail e escopo. Equipes.agendar_consulta registra solicitante e momento da próxima tentativa, sem fazer rede nessa requisição. A migração 5 acrescenta solicitada, solicitante e codigo à tabela tarefas.

Escritorio.consultas_tick processa solicitações avulsas e periódicas em sequência, respeitando reserva e espera. Após erro transitório, a espera cresce e considera Retry-After. Após cinco falhas consecutivas, o ciclo pausa. Falhas de credencial ou configuração exigem atenção, sem repetição automática indefinida. O sucesso limpa o pedido e reaproveita a versão quando não há diferença. Um reinício preserva a fila no banco.

O frontend mostra fila e última consulta válida separadamente. Pode ler o estado local a cada 15 segundos sem consultar o CNJ, evitando recarregar durante edição ou histórico aberto. Os dados anteriores continuam disponíveis e um processo ainda sem resposta não recebe conteúdo inventado. O servidor precisa permanecer ligado e com worker ativo. Veja fila-atualizacoes-datajud.md para o roteiro e seus limites.

11 fontes.py e comparação das informações

validar_numero remove pontuação e espaços, exige vinte dígitos, confere o segmento e tribunal suportados e calcula os dígitos verificadores. Um número com formato válido não comprova que o processo existe, que está público ou que está disponível naquela fonte.

serializar produz JSON com ordenação de chaves e separadores estáveis. Isso reduz diferenças causadas apenas pela forma de representar um objeto. versao calcula SHA-256 dessa representação. Hash de conteúdo ajuda a identificar igualdade; ele não comprova autenticidade jurídica da fonte.

normalizar valida o contrato mínimo: objeto com registros, registros com lista de movimentos e movimentos com código e data textual. Remove movimentos exatamente repetidos e ordena os resultados de maneira

estável. Ela não interpreta o significado jurídico de uma movimentação, nem garante que uma data textual seja um prazo.

diferencas monta conjuntos de movimentos considerando o grau do registro, encontra inclusões e remoções e compara dados cadastrais separados dos movimentos. O retorno classifica primeiro registro, sem diferença, nova movimentação, correção ou remoção e alteração cadastral. Uma remoção pode ser correção da fonte; a interface não deve anunciar uma conclusão jurídica automática.

DataJudValidado reaproveita DataJud e acrescenta validação do número e normalização. Demonstracao devolve conteúdo fictício controlado por etapa. O primeiro estado tem uma distribuição; o segundo acrescenta uma audiência fictícia. Trocar a fonte por uma demonstrativa nunca pode ser uma recuperação silenciosa de erro real.

12 equipes.py e o trabalho compartilhado

Equipes.__init__ cria tabelas de organização e acrescenta colunas ao cadastro antigo de processos. _org associa uma conta antiga a um escritório pessoal se ela ainda não tiver vínculo. Isso preserva contas e casos anteriores. Uma migração muda a estrutura do banco; não é o mesmo que importar uma carteira de processos.

org verifica associação e, quando necessário, papel de administrador. caso encontra o processo e verifica acesso ao escritório dele. Esses métodos são pontos importantes do isolamento: um ID de processo não é uma autorização. contexto lista os escritórios aos quais a pessoa pertence; membros informa integrantes do escritório autorizado.

adicionar valida CNJ e apelido, recusa duplicação dentro do escritório e cria o processo com responsável. alterar permite organizar apelido, etiquetas, observações, responsável, arquivamento e periodicidade. O responsável precisa ser membro. Um intervalo não nulo precisa respeitar o mínimo de 900 segundos. Arquivar muda a seleção de trabalho, não apaga o histórico.

consultar começa com uma transação curta que verifica acesso, arquivamento e reserva. A reserva lease impede duas consultas simultâneas de ocuparem a mesma tarefa. A chamada à rede ocorre depois dessa transação. Ao receber a fonte, outra transação relê o estado, compara, grava versão e consulta, atualiza referência e agenda a próxima tentativa.

Quando o conteúdo é idêntico, a versão atual é reutilizada. Quando o conteúdo volta a um estado antigo após uma mudança intermediária, uma nova transição recebe nova versão. Isso evita tratar uma leitura antiga como se alguém tivesse lido o retorno recente da fonte.

Se houver novidade, notificacoes registra o evento do escritório e emails recebe mensagens para membros que aderiram à preferência. Se ocorrer falha, consultas registra o erro, tarefas recebe tentativa e próxima execução e casos conserva o snapshot anterior com a informação da falha. É necessário mostrar a falha na interface para não transmitir falsa atualidade.

revisar registra pessoa, versão, instante e observação. detalhe reúne versões e conferências para a linha do tempo. historico é herdado da base e usa a verificação de caso sobrescrita. audit registra ações de organização e trabalho. Auditoria operacional não significa assinatura digital nem trilha inviolável: um operador com acesso ao arquivo SQLite tem poderes que a interface não tem.

convidar gera um token de 24 horas vinculado a e-mail e escritório; aceitar confere destinatário e validade e consome o convite. Na classe Piloto, ambos exigem e-mail confirmado. revogar tem uma versão antiga neste arquivo e uma versão efetiva em Escritorio, com reatribuição de responsabilidades.

importar lê CSV separado por ponto e vírgula, valida até 500 linhas, identifica duplicados e permite prévia sem gravação. A confirmação processa as linhas válidas e informa rejeições. Não é uma transação única de tudo ou nada; a interface precisa apresentar o resultado por linha. Um processo inválido não deve ser anunciado como importado.

tick e email_tick deste arquivo são legados da primeira versão de equipes. A aplicação atual usa consultas_tick e email_tick de Escritorio em ciclos separados. backup_tick continua herdado e usa o utilitário consistente de cópia. Saber qual implementação é efetivamente chamada evita estudar um método legado como se ele fosse o caminho atual.

13 rotina.py e as providências

Escritorio cria providencias e eventos_providencia e acrescenta os campos de reserva da fila de e-mails. ESTADOS enumera os estados aceitos. TRANSICOES informa os caminhos permitidos, impedindo saltos arbitrários de uma atividade recém-criada para concluída.

criar_providencia valida título, descrição, prioridade, responsáveis e data interna opcional. Confere se a versão pertence ao processo e se responsável e revisor pertencem ao escritório. Grava o estado inicial e um evento. A providência pode ser geral ou ligada a uma versão específica que motivou a ação.

rotina consulta as atividades com filtros minhas, equipe, atrasadas, revisar e concluídas. Faz busca e paginação de 25 resultados no banco. O resumo informa contagens de trabalho. Uma atividade sem data não é vencida. A data interna é um compromisso administrativo; não há calendário de feriados ou contagem de prazo processual.

alterar_providencia verifica acesso, participante autorizado, revisão apresentada e transição de estado. A revisão é um inteiro incrementado a cada alteração. Se duas pessoas abrem a revisão 1, a primeira salva e cria revisão 2. A segunda recebe conflito 409 ao tentar gravar revisão 1. Isso evita perda silenciosa de trabalho.

Enviar para revisão exige evidência textual. Concluir exige revisor ou administrador e mantém a exigência de evidência. Reabrir exige comentário. A mesma pessoa pode ser executor e revisor para uso individual; separação obrigatória entre duas pessoas ainda não é uma política configurável. historico_providencia mostra eventos autorizados do registro.

consultas_tick busca até três tarefas elegíveis e usa ThreadPoolExecutor para consultas paralelas limitadas. Não cria uma thread ilimitada por processo. A reserva da consulta continua sendo o controle individual. O limite global de consultas manuais da API não é, por si só, um limitador distribuído de todos os workers.

email_tick reserva uma mensagem dentro de transação curta, revalida membro e preferência, faz o envio fora da transação e grava resultado depois. Uma chamada de rede lenta não deve manter o SQLite com uma transação de escrita aberta. A reserva expirada permite recuperação após interrupção.

Ainda existe uma janela de incerteza externa: o provedor pode aceitar a mensagem e o processo cair antes de registrar esse aceite. Uma retentativa pode duplicar o envio. O sistema não promete exatamente uma entrega externa. Para melhorar essa garantia, o provedor precisa oferecer um mecanismo compatível de idempotência e acompanhamento.

operacao lista diagnóstico administrativo de consultas e e-mails recentes. reprocessar confere administração e elegibilidade antes de agendar nova tentativa. revogar remove o membro, transfere processos e funções para o administrador e cancela mensagens pendentes. Uma mensagem que já começou a ser transmitida não pode ser desfeita retroativamente.

14 piloto.py e as melhorias desta revisão

Piloto é a classe concreta criada pela API atual. Seu construtor executa as bases e acrescenta a evolução de identidade e carteira. migrar_carteira procura a restrição antiga UNIQUE(usuario, numero, tribunal). Antes de reconstruir a tabela, cria uma cópia consistente com nome exclusivo no mesmo diretório.

A reconstrução copia todas as colunas e conserva os IDs, troca a unicidade para escritório, número e tribunal e verifica referências com PRAGMA foreign_key_check antes do commit. Se houver conflito ou vínculo inválido, ocorre rollback. O backup local da migração não substitui uma cópia externa protegida. A operação deve acontecer com inicialização controlada, sem iniciar simultaneamente várias instâncias migradoras.

conta devolve somente ID, e-mail e instante de confirmação. solicitar_verificacao gera segredo aleatório, guarda hash e expiração de trinta minutos e entrega o código pelo adaptador de correio. Se o envio falhar, remove o token correspondente e propaga o erro. Não devolve código de confirmação na resposta da API.

confirmar_email exige token pertencente à conta autenticada, válido e ainda não consumido. Atualiza email_verificado e elimina os tokens da conta em transação. Contas antigas não são marcadas como verificadas automaticamente. exigir_email bloqueia compartilhamento e consulta real sem essa confirmação; treino e demonstração permanecem disponíveis.

Em desenvolvimento, a mensagem fica em um arquivo local. Ler esse arquivo ensina o fluxo, mas não prova que a pessoa controla uma caixa de e-mail externa. Em produção, a comprovação depende de entrega pelo canal configurado. A confirmação de e-mail não é segundo fator de autenticação em cada login.

carteira recebe escritório, página, busca e filtro. Monta condições SQL parametrizadas, conta resultados e devolve vinte itens da página, indicadores completos e referência à demonstração. Não depende da lista legada limitada a mil registros. A busca usa LIKE e pode precisar de índices ou busca especializada se a carteira crescer muito.

resumo_diario calcula providências do usuário, vencidas e aguardando revisão. solicitar_dados registra um pedido administrativo de retenção ou exclusão; não executa eliminação. exportar_escritorio organiza dados relacionados do escritório em JSON, sem hashes de senha, sessões, convites ou tokens. É exportação administrativa, não um formato de restauração já implementado.

15 consulta.py correio.py backup.py e scripts de execução

consulta.py é o exemplo mais direto para aprender Requests e Beautiful Soup. validar_entradas exige de um a três casos, códigos no formato do catálogo e referência anterior textual. consultar_fonte faz GET da fonte HTML local, com timeout e sem depender do proxy do sistema, verifica HTTP e chama extrair_e_comparar.

extrair_e_comparar usa seletores CSS para encontrar article[data-case], .movement e .date. Extrai texto e compara com previous. Um registro ausente recebe ok falso e erro; não vira sucesso fictício. main usa argparse para permitir execução pelo terminal e grava um relatório JSON. A data da fonte ausente continua sendo uma informação incompleta mesmo quando o usuário registra que leu.

correio.py concentra entregar. Em desenvolvimento, grava JSON em output/email-local. Em produção, exige configurações SMTP, inicia STARTTLS, autentica e envia EmailMessage. O retorno aceito_pelo_smtp comprova apenas aceite pelo servidor, não leitura ou entrega final. Nenhum provedor específico de eventos de entrega está integrado nesta versão.

backup.py oferece copiar e interface de terminal. Abre origem em modo somente leitura, executa integrity_check, usa o mecanismo backup do SQLite e verifica o destino. Recusa sobrescrever arquivo existente. Restaurar significa apontar uma execução isolada para uma cópia verificada, não substituir às cegas um banco em uso.

rpa_navegador.py contém o roteiro Playwright: inicia servidor de laboratório, abre navegador, adiciona caso fictício, consulta, compara, registra conferência, baixa CSV e grava evidências. Requests realiza HTTP sem navegador; Beautiful Soup interpreta HTML recebido; Playwright opera uma página como usuário. O roteiro não equivale a integração com sistemas judiciais reais e não foi executado nesta revisão, porque o acesso ao navegador local está desativado.

servidor.py implementa o servidor legado com ThreadingHTTPServer e Handler. Ele é útil para estudar requisições sem FastAPI e para o roteiro histórico, mas não é a entrada do piloto. iniciar_datajud.py também é legado: busca uma chave pública na documentação e chama o servidor antigo. Não o use como lançador do aplicativo atual. AMBAR_DATAJUD_HABILITADO e DATAJUD_API_KEY pertencem à configuração explícita da aplicação atual.

16 As páginas da interface

index.html é a apresentação pública. Explica o produto e separa conhecer, treinar, criar acesso e usar. Não é o painel de processos e não deve concentrar todos os formulários. As imagens office.jpg e workspace.jpg são recursos visuais; não representam dados de processos. marca.svg é um recurso de marca legado; o cabeçalho atual usa composição tipográfica.

laboratorio.html é o treino sem conta, com os cenários cível, contratual e trabalhista fictícios. portal.html é a fonte HTML controlada usada pelo treino. Esses códigos de treino não são números CNJ de processos reais. runtime-config.js indica o modo disponível quando servido estaticamente; o FastAPI fornece sua versão dinâmica na mesma rota.

acesso.html contém formulário de cadastro e login em modos separados. acesso.js alterna o modo, orienta os campos, evita reenvio simultâneo, faz POST, apresenta erro e encaminha para acompanhamento.html após sucesso. O guia da área de trabalho pode ser solicitado após o acesso. A senha é limpa do formulário e não é persistida em localStorage pelo código.

acompanhamento.html estrutura a área persistente: entrada, escritório, processos, rotina, equipe, notificações, histórico e providência. Parte dos painéis novos é inserida por piloto.js. Cada botão precisa ter ação, estado de carregamento, erro e possibilidade de recuperação. Só apresentar uma tela bonita sem lidar com falhas tornaria o fluxo incompleto.

amber.css estiliza o laboratório original. acompanhamento.css contém a identidade e os componentes da aplicação de equipes. percurso.css conecta apresentação, acesso e treino e ajusta diálogos. CSS decide apresentação, não autorização. Alterar uma classe não pode liberar uma ação que o servidor proíbe.

17 acompanhamento.js e a comunicação da aplicação

el encontra um elemento pelo ID. text cria um elemento e atribui textContent, evitando interpretar texto recebido como HTML. button e field constroem controles consistentes. status informa andamento, sucesso ou falha. signedIn alterna a apresentação autenticada, mas a API continua responsável pelo acesso real.

api centraliza fetch, método, JSON, timeout, verificação de tipo de resposta e tratamento de 401. eventosDaAcao guarda caminhos de ações concluídas para o tutorial; não inclui corpos com senha ou token. run evita duas ações simultâneas, marca áreas como ocupadas, trata erro e restaura controles e foco ao terminar.

load obtém escritórios, mantém uma seleção válida, carrega carteira paginada e membros, atualiza os painéis e busca notificações. renderMetrics usa as contagens completas devolvidas pelo servidor. render desenha apenas os itens da página e os controles de paginação. A busca espera um pequeno intervalo antes de enviar nova consulta, evitando chamada a cada tecla imediata.

caseCard monta identidade, fonte, estado, última consulta válida, prévia e ações do processo. history carrega histórico técnico e versões, apresenta diferenças e permite leitura ou criação de providência. review registra a leitura de uma versão concreta. editor permite organizar responsável, etiquetas, observações, periodicidade e arquivamento.

submit converte FormData em objeto e encaminha para a ação correspondente. Os eventos de cadastro, importação, equipe e recuperação usam esse padrão. A exportação CSV trata download de um arquivo por Blob e revoga a URL temporária. iniciarAplicacao evita que a inicialização aconteça duas vezes enquanto os scripts do guia terminam de carregar.

Os scripts compartilham funções no mesmo contexto de página; a ordem de carregamento importa. Acompanhamento vem antes de rotina, piloto e guia. Esse desenho é simples para o projeto atual, mas uma evolução com módulos JavaScript poderia tornar dependências mais explícitas. Não mude a ordem sem testar a inicialização e o tutorial.

18 rotina.js piloto.js e guia.js

rotina.js cria o seletor de CSV, abre o formulário de providência e gerencia a área Minha rotina. atualizarRotinaContexto limpa dados quando o escritório muda e aplica visibilidade de administração. trocarArea alterna processos e rotina. abrirProvidencia preenche processo, versão e participantes. carregarRotina consulta filtros e paginação do servidor.

providenciaCard desenha a tarefa, os estados permitidos, comentário, evidência e histórico. A API ainda verifica tudo o que a tela oferece. carregarOperacao mostra filas e reprocessamento para administradores. Um erro 409 significa atualizar a referência antes de insistir no envio; não deve ser escondido como sucesso.

piloto.js acrescenta Minha conta, confirmação de e-mail, resumo diário e Dados do escritório. O formulário de confirmação envia token apenas no corpo do POST. A exportação baixa JSON autenticado. A solicitação de dados mostra expressamente que não apaga registros. carregarPainelPiloto atualiza identidade, contagens e visibilidade conforme o papel da pessoa no escritório.

guia.js descreve etapas da área autenticada e relaciona uma orientação com um controle existente e, quando aplicável, uma ação bem-sucedida da API. Guarda apenas passo e estado de atividade no navegador. pausarGuia remove destaques; iniciarGuia retoma ou reinicia; mostrarGuia apresenta instrução e alvo; avancarGuia progride. Pausar o guia não encerra a sessão.

O guia não deve substituir o design normal. Uma pessoa precisa conseguir usar o aplicativo sem seguir todas as etapas. Não preencha ou envie convites, exclusões ou dados reais apenas para avançar uma instrução. Ações opcionais de demonstração devem ser distinguidas de ações com efeitos no escritório.

19 amber.js e o laboratório de treino

amber.js mantém catálogo, seleção, referências e histórico da sessão de treino. visibleCases aplica busca e filtro local porque o catálogo fictício é pequeno. statusOf transforma o estado de um caso em texto. render e renderCatalog atualizam lista e catálogo. openDetail mostra antes e depois; closeDialog fecha o diálogo e coordena o guia.

carregarFonteTreino busca portal.html, verifica sucesso e extrai campos conhecidos com DOMParser, construindo novos elementos com conteúdo textual. Não injeta o HTML completo da fonte no aplicativo. A tentativa anterior de usar iframe foi retirada porque a política frame-ancestors proíbe incorporação. A correção preservou essa proteção e tornou a visualização interna independente do iframe.

O carregamento tem identificação de requisição para ignorar respostas antigas sobrepostas. A etapa da fonte avança apenas após carregamento válido. Falha mostra erro e permite tentar novamente. A fonte também pode ser aberta separadamente. Isso ensina um princípio de interface: o próximo passo só deve depender de uma ação cuja conclusão foi realmente observada.

configurarCasoGuiado adapta instruções ao cenário selecionado. advance e updateCoach coordenam eventos e apresentação do guia. O cenário trabalhista inclui informação incompleta para ensinar a diferença entre registrar leitura e resolver a ausência de dados. A lista e o histórico desse laboratório não são a carteira persistente do escritório.

20 Banco de dados e relações

usuarios guarda identidade e hash de senha; sessoes guarda hashes de tokens e expiração; verificacoes_email e recuperacoes guardam tokens de uso temporário em forma de hash. organizacoes e membros definem escritórios e papéis. convites guarda destino, escritório e expiração. Esses registros são de controle de acesso, não de movimentação judicial.

casos liga um processo a escritório e responsável e guarda referência atual, versão, periodicidade e erro. consultas registra cada tentativa e seu resultado. versoes guarda conteúdo normalizado e diferenças. conferencias liga uma pessoa à versão lida. notificacoes liga a novidade ao escritório. tarefas controla quando consultar novamente e quem reservou a execução por tempo limitado.

providencias guarda estado atual, pessoas, evidência e revisão. eventos_providencia guarda a sequência de ações. auditoria registra ações relevantes do escritório. preferencias controla adesão aos avisos; emails registra a fila. limites controla contadores temporários. migracoes registra evolução do schema. solicitacoes_dados guarda pedidos administrativos pendentes.

Chave primária identifica uma linha. Chave estrangeira relaciona uma linha com outra. UNIQUE impede combinações repetidas. Índice melhora consultas compatíveis com suas colunas. Uma transação agrupa alterações que precisam ser confirmadas ou desfeitas juntas. SQLite permite diversos leitores, mas a escrita requer cuidado com concorrência e duração da transação.

Nunca edite manualmente IDs e vínculos de um banco de uso real para “consertar a tela”. Reproduza o problema em cópia, escreva uma migração e teste a preservação. O arquivo de backup deve ser protegido como o banco original, porque contém as mesmas informações sensíveis.

21 Percursos completos para acompanhar no código

Cadastro e entrada: acesso.js envia email e senha -> aplicacao.dispatch aplica limite -> Caderno.autenticar valida e grava -> resposta define cookie -> load consulta contexto -> API resolve usuário -> organização e carteira são carregadas. O hash da senha e o token de sessão têm finalidades distintas.

Confirmação: piloto.js solicita -> API limita -> Piloto gera segredo e hash -> correio entrega instruções -> usuário envia token autenticado -> confirmar_email valida conta e validade -> banco registra confirmação e consome token. No modo local, a entrega é arquivo; em produção, depende de SMTP real.

Consulta: caseCard chama consultar -> API verifica sessão e limite -> Piloto exige e-mail se fonte real -> Equipes reserva tarefa -> fonte retorna -> normalizar e diferencas processam -> transação grava versão, referência, consulta e fila -> frontend recarrega a carteira. Em erro, histórico e falha são registrados e o snapshot é preservado.

Providência: processo ou versão abre formulário -> rotina.js envia responsáveis, título e contexto -> Escritorio valida pertencimento -> banco grava providência e evento -> responsável inicia -> evidencia resultado -> revisor aprova -> evento e revisão são incrementados. O banco e a API não dependem de um botão estar visível para proteger o fluxo.

Importação: arquivo CSV é lido no navegador -> conteúdo é enviado para prévia -> servidor verifica linhas e duplicados -> pessoa revisa -> confirmação revalida e processa -> relatório distingue aceitas e rejeitadas. Uma mudança no conteúdo invalida a aprovação anterior da prévia.

22 Segurança e limites da versão

O projeto contém validação de entrada, hashes, sessões com expiração, autorização por escritório, proteção de origem, política de conteúdo e limites de algumas operações. Isso reduz riscos específicos; não equivale a certificação de segurança. A separação entre escritórios precisa ser testada em cada nova rota, inclusive arquivos e exportações.

A aplicação ainda não tem MFA, política executável de exclusão/retencão, armazenamento privado de documentos ou isolamento granular de cada processo entre membros do mesmo escritório. Membros têm acesso ao espaço da equipe conforme o desenho atual. Um escritório que precise de compartimentos confidenciais exige um modelo de autorização adicional.

Os backups automáticos atuais são locais e diários quando habilitados. Não há rotação criptografada externa pronta. A fila não inclui confirmação de entrega do provedor. O worker depende do processo estar rodando. Não existe homologação de dados reais, provedor, domínio e infraestrutura nesta revisão. Os testes de DOM não avaliam visualmente páginas.

O novo registro de solicitação de exclusão é uma entrada administrativa, não um mecanismo que elimina dados e backups. O novo parâmetro AMBAR_WORKER não é um sistema distribuído de eleição de líder. O JSON administrativo é exportação, não um restaurador. Essas distinções precisam aparecer na apresentação para não atribuir ao sistema garantias que ele ainda não oferece.

23 Testes e o que cada arquivo comprova

test_consulta.py testa entrada, extração HTML, comparação, falha de fonte e resposta do laboratório. test_acompanhamento.py testa a base pessoal e o servidor legado, incluindo sessão e isolamento inicial. Eles preservam o aprendizado e as regressões da primeira versão; não cobrem sozinhos a aplicação de equipes.

test_equipes.py testa CNJ, migração, convite, isolamento, conferência por versão, referência preservada, reserva concorrente, importação, fila e restauração. test_rotina.py testa executor e revisor, conflito de revisão, reprocessamento, envio fora de transação e revalidação de acesso. test_correio.py usa simulação de SMTP e verifica que aceite não significa entrega.

test_piloto.py testa o token de confirmação, conta correta, expiração, uso único, exigência de e-mail nos convites, reconstrução preservadora, duplicação por escritório, carteira com 1.005 registros, exportação sem segredos e solicitação sem apagamento automático. test_api_v2.py sobe FastAPI com banco temporário e faz chamadas HTTP reais para sessões, rotina e rotas de piloto.

tests/ui.test.cjs simula o DOM com jsdom e respostas de rede controladas, exercitando escritório, versões, rotina, importação, guia e conta. tests/percurso.test.cjs testa separação da apresentação e modos de acesso. tests/treino.test.cjs percorre os três cenários e recupera falha ao carregar a fonte. Nenhum desses arquivos executa um navegador visual real.

Execute Python com um ambiente que contenha requirements.txt e use python -m unittest discover -s backend -v. Execute npm ci quando necessário e depois npm run test:ui. Testes usam bancos temporários; não devem apontar para output/ambar.sqlite3 de uso pessoal. Registre resultado, data, ambiente e limites. Um teste com mock comprova sua regra sob respostas controladas, não o funcionamento atual do

provedor externo.

Na revalidação de 05/11/2026, passaram 56 testes Python e 16 testes JavaScript. As chamadas HTTP da suíte são locais e SMTP é simulado. O ensaio externo do DataJud e a demonstração visual ficam em relatórios separados e não homologam uma carteira empresarial. A interface foi operada no navegador local, incluindo histórico, providências, fila e laboratório fictício. A preparação para publicação no GitHub inclui README e capturas públicas; o banco e as credenciais permanecem locais.

24 Como executar e investigar um problema

Use uma versão de Python compatível com as dependências e crie .venv com py -m venv .venv. Instale requirements.txt com o Python dessa pasta e inicie backend/aplicacao.py. No Windows, INICIAR.cmd automatiza esses comandos. Se a porta estiver ocupada, identifique o processo antes de encerrar qualquer aplicativo.

Abra http://127.0.0.1:8093/index.html para apresentação, /laboratorio.html?treino=1 para treino e /acompanhamento.html para área persistente. Ao configurar outra porta, ajuste AMBAR_PORT e AMBAR_ORIGIN coerentemente. Não misture localhost e 127.0.0.1 ao testar cookies e origem sem compreender as diferenças.

Se uma ação falhar, comece pela mensagem da tela e pelo identificador da requisição. Na aba Network do navegador, veja método, caminho, status e resposta. No terminal do servidor, procure o erro correspondente. Em um banco copiado, confirme se a conta, organização, versão ou revisão esperada realmente existe.

Erro 401 pede login. Erro 403 exige conferir permissão, origem ou e-mail confirmado. Erro 409 pede examinar duplicação, reserva ou revisão desatualizada. Erro 503 pode indicar fonte ou SMTP desativado. Resposta HTML onde o frontend espera JSON frequentemente indica caminho ou servidor errado. Não resolva isso silenciando a validação de resposta.

25 Trilha prática de estudo em dez sessões

Sessão 1: execute apresentação e treino e explique a diferença para a carteira persistente. Abra HTML e localize os IDs usados pelo JavaScript. Exercício: altere um texto de ajuda, recarregue e identifique qual arquivo o forneceu.

Sessão 2: estude funções, listas, dicionários, None, condições e exceções. Reescreva validar_entradas com mensagens próprias em arquivo de exercício. Teste uma lista vazia, um código repetido e um caso válido. Explique por que rejeitar entrada evita erros posteriores.

Sessão 3: execute extrair_e_comparar com um HTML pequeno. Retire a data e observe o resultado. Troque o texto da movimentação. Explique como o seletor CSS se relaciona com uma tag HTML e por que uma mudança de estrutura externa pode quebrar extração.

Sessão 4: leia fontes.py e crie duas listas fictícias de movimentos. Teste inclusão, repetição, remoção e mudança de grau. Explique quais alterações geram diferença e quais apenas mudam a ordem. Não consulte processos reais para fazer esse exercício.

Sessão 5: estude SQLite em banco temporário. Crie uma tabela simples, insira dados parametrizados e provoque uma exceção dentro de uma transação. Confirme rollback. Compare com o fluxo de gravação de uma versão.

Sessão 6: percorra cadastro, cookie e usuario. Explique hash de senha, salt, token de sessão e expiração com suas próprias palavras. Não exiba tokens reais na apresentação. Teste saída e tentativa de consulta sem sessão.

Sessão 7: crie duas contas e dois escritórios de teste. Confirme e-mails pelo canal local, crie convite, aceite e revogue. Tente acessar o escritório errado. Observe a diferença entre não mostrar um botão e receber uma recusa da API.

Sessão 8: crie uma providência, inicie execução, envie evidência e revise. Abra o mesmo registro em duas sessões de teste e provoque conflito de revisão. Explique por que a segunda gravação não pode substituir a primeira silenciosamente.

Sessão 9: estude filas, leases e retentativas. Faça uma fonte de teste falhar e preserve a referência. Leia o teste que grava no banco durante o mock de envio e explique por que isso detecta uma transação longa durante a rede.

Sessão 10: faça backup e abra uma cópia isolada. Demonstre o fluxo completo em cinco minutos e responda às perguntas seguintes sem ler. Só apresente como domínio pessoal as partes que conseguir explicar, reproduzir e alterar.

26 Perguntas para treinar a apresentação técnica

O que o projeto automatiza? Resposta esperada: consulta e comparação de informações, registro de versões e organização de trabalho; a interpretação e a conferência permanecem humanas.

Por que Python? Neste projeto, ele reúne HTTP com Requests, extração com Beautiful Soup, API com FastAPI e ferramentas de automação e persistência com bibliotecas adequadas ao aprendizado. Isso descreve a escolha local, não significa que Java seja incapaz de fazer o mesmo.

Onde entra Playwright? No roteiro de operação da interface. A consulta DataJud usa Requests e JSON. O laboratório HTML usa Requests e Beautiful Soup. Não atribua as três ferramentas a uma mesma função se o código não as chama naquele caminho.

Como evita misturar escritórios? A sessão identifica usuário; o serviço valida associação e papel usando org e caso antes de acessar ou alterar dados. Testes de acesso cruzado precisam acompanhar qualquer nova rota. Filtros no frontend não oferecem essa garantia.

Como sabe que mudou? Normaliza a resposta, organiza os movimentos e compara conteúdos por grau, incluindo inclusões, remoções e cadastro. Uma versão nova não herda automaticamente a conferência de outra transição. O aplicativo mostra evidências, não decide o significado jurídico.

O que acontece se a fonte cair? Registra a tentativa e o erro, preserva a referência válida e agenda nova tentativa limitada. A tela precisa deixar claro que os dados exibidos são da consulta anterior. O sistema não inventa uma atualização para manter aparência de funcionamento.

Já está pronto para um escritório usar sem acompanhamento? Ainda não. Há fluxo local e testes, mas faltam homologação de fonte e infraestrutura, validação visual, MFA e políticas efetivas de dados, entre outros itens do entregável. A proposta responsável é um piloto controlado após resolver os bloqueios aplicáveis.

27 Referências para aprofundamento

Python tutorial: https://docs.python.org/3/tutorial/ . Consulte funções, estruturas, classes, exceções e módulos enquanto acompanha os exemplos locais.

SQLite em Python: https://docs.python.org/3/library/sqlite3.html . Estude conexões, parâmetros e transações. FastAPI: https://fastapi.tiangolo.com/ . Estude requisições, dependências, validação e implantação. Requests: https://requests.readthedocs.io/en/latest/ . Estude Session, timeout, erros e JSON.

Beautiful Soup: https://www.crummy.com/software/BeautifulSoup/bs4/doc/ . Estude seletores e extração de texto. Playwright Python: https://playwright.dev/python/docs/intro . Estude locators, assertions, rastreamento e isolamento de contextos. DataJud: https://datajud-wiki.cnj.jus.br/api-publica/acesso/ . Confirme sempre as condições vigentes antes de habilitar uso real.

Segurança entre organizações: https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html . Use como referência de revisão; a existência de controles no projeto não significa conformidade completa com a orientação.

28 O que você deve conseguir explicar ao terminar

Você deve conseguir partir de um clique e indicar o evento JavaScript, a rota HTTP, a sessão, a autorização, o método Python, a consulta SQL, a fonte envolvida e a resposta apresentada. Também deve conseguir indicar o que foi testado, onde há simulação e quais limitações permanecem.

O próximo passo de aprendizado não é memorizar todo o repositório. É reproduzir um fluxo pequeno sem copiar, modificar uma regra, escrever um teste que falhe antes da correção e explicar por que a mudança preserva os outros percursos. Use o catálogo complementar para localizar cada símbolo e este manual para entender a relação entre eles.

29 Guia das rotas da aplicação atual

Todas as rotas abaixo começam por /api. As operações de escritório exigem sessão, e as administrativas exigem também papel apropriado. Um caminho aceito não significa que qualquer usuário pode executá-lo. GET usa parâmetros na URL; POST envia um objeto JSON. A lista descreve o contrato local atual e complementa o roteador genérico, que ainda não expõe contratos individuais completos em OpenAPI.

GET /saude: verifica acesso básico ao banco e devolve status, engine, version e indicação da flag de fonte. Não verifica o provedor externo. POST /consultar: recebe cases com id e previous para o laboratório HTML. É desativada em produção e não é a consulta persistente de um processo CNJ.

POST /conta/cadastro e POST /conta/login: recebem email e senha e definem cookie de sessão. POST /conta/sair revoga o cookie e a sessão apresentada. POST /conta/recuperar recebe email e responde de maneira genérica para não informar diretamente se a conta existe. POST /conta/redefinir recebe token e senha, consome a recuperação e revoga sessões.

GET /conta: devolve id, email e email_verificado da própria pessoa. POST /conta/verificacao/solicitar não recebe código; gera um novo e entrega pelo correio configurado. POST /conta/verificacao/confirmar recebe token e exige que ele pertença à conta autenticada. Não use query string para enviar senha ou token, porque URLs podem aparecer em histórico e logs.

GET /contexto: devolve os escritórios e papéis disponíveis. GET /me?org=ID: devolve identidade mínima após conferir acesso ao escritório. POST /organizacao/nome recebe org e nome e exige administração. GET /membros?org=ID lista integrantes. POST /membros/revogar recebe org e usuario e transfere responsabilidades na implementação atual.

POST /convites/criar recebe org e email, exige administrador com e-mail confirmado e devolve token destinado ao convidado. POST /convites/aceitar recebe token e exige conta com e-mail confirmado correspondente ao convite. O token de convite é uma capacidade temporária de ingresso; não deve ser divulgado publicamente.

GET /carteira recebe org, pagina, busca e filtro. filtro aceita all, pending, failed e archived. O resultado possui itens, total, pagina, por_pagina, resumo e demonstracao. Os indicadores do resumo são calculados sobre o escritório inteiro; total representa a seleção atual. A página contém até vinte processos.

GET /casos é compatibilidade legada e continua limitado. POST /casos recebe org, numero, tribunal e apelido para criar um processo. O frontend atual lê por /carteira, mas cria por /casos. POST /demonstracao recebe org e retorna o identificador do exemplo fictício; a criação de demonstração pela API é desativada em produção.

POST /casos/ID/consultar consulta uma referência persistente e devolve estado, dados, diferenca e versao_id. POST /casos/ID/avancar-demo altera a etapa fictícia e só funciona na demonstração fora de produção. GET /casos/ID/historico devolve tentativas. GET /casos/ID/detalhe devolve versões e conferências. POST /casos/ID/revisar recebe versao_id e observacao opcional para registrar leitura.

POST /casos/ID/alterar aceita organização interna do processo: apelido, etiquetas, observacoes, responsavel, arquivado e intervalo. Ele não altera dados no tribunal. Intervalo zero desativa agendamento; os demais precisam respeitar o mínimo. O servidor verifica associação do responsável e acesso ao processo.

GET /rotina recebe org, filtro, pagina e busca. filtro aceita minhas, equipe, atrasadas, revisar e concluidas. O resultado contém itens, total, pagina e resumo. POST /providencias recebe caso, versao opcional, titulo, descricao, responsavel, revisor, data_interna opcional e prioridade. A organização é obtida do próprio processo, não confiada a um campo arbitrário do visitante.

POST /providencias/ID/alterar recebe revisao e os campos de alteração, como estado, comentario e evidencia. Um número de revisão antigo recebe conflito. GET /providencias/ID/historico lista eventos autorizados. O formato interno dos estados usa a_fazer, em_andamento, aguardando, em_revisao e concluida; os textos da tela podem ser diferentes e mais legíveis.

POST /importar recebe org, csv e confirmar. Sem confirmação, devolve prévia. GET /exportar?org=ID gera CSV da carteira completa. GET /dados/exportar?org=ID exige administrador e devolve JSON com histórico

relacionado. POST /dados/solicitar recebe org, tipo e motivo, registrando pedido de exclusao ou retencao sem executar eliminação.

GET /notificacoes?org=ID lista avisos recentes; GET /auditoria?org=ID exige administração e lista ações registradas. POST /preferencias recebe email booleano para adesão aos avisos. GET /operacao?org=ID é diagnóstico administrativo. POST /operacao/reprocessar recebe org, tipo e id e valida se a tarefa pode ser reprocessada.

30 Leitura comentada de pequenos trechos

Parâmetros SQL e o formato da tupla

```
with self.conexao() as db:
    linha = db.execute(
        'SELECT id,email FROM usuarios WHERE id=?',
        (usuario,)
    ).fetchone()
```

A primeira linha abre um contexto que fecha a conexão ao sair. A segunda inicia uma operação de leitura. O texto SQL define estrutura e seleção; o valor de usuario é entregue separadamente na tupla. O driver trata esse valor como dado. fetchone pede um resultado. Se não houver registro, linha será None e deve ser tratada antes de acessar linha['email'].

Trocar (usuario,) por uma interpolação de texto é uma mudança de segurança, não uma simplificação inocente. Trocar fetchone por fetchall também muda o contrato: o chamador passa a receber uma lista em vez de um registro. Ao modificar, procure todos os chamadores e atualize testes.

Token de confirmação e armazenamento

```
segredo = secrets.token_urlsafe(32)
digest = hashlib.sha256(segredo.encode()).hexdigest()
```

segredo é um valor aleatório para ser entregue ao destinatário pelo canal adequado. encode transforma texto em bytes. sha256 calcula um resumo desses bytes. hexdigest representa o resultado como texto hexadecimal para armazenamento. O banco recebe digest, não segredo. Ao confirmar, a aplicação calcula o hash do token apresentado e procura a correspondência válida.

O hash sozinho não implementa expiração: por isso o registro tem expira. Também não implementa uso único: por isso o registro é eliminado após sucesso. A vinculação a usuario impede que outra conta autenticada consuma o token. A mensagem local do ambiente de estudo contém o segredo porque simula a mensagem recebida; ela não deve ser publicada no repositório.

Uma transição que pode falhar

```
if dados.get('revisao') != p['revisao']:
    raise FalhaAPI(409, 'Esta providência mudou.')
```

dados representa a alteração enviada. p é o registro atual do banco. A condição compara a revisão apresentada com a atual. Quando são diferentes, raise interrompe antes da gravação. O frontend deve mostrar conflito e pedir atualização. Apenas repetir o POST automaticamente com a revisão nova pode esconder uma mudança que a pessoa ainda não leu.

Um campo opcional bem definido

```
valor_data = dados.get('data_interna', '')
data = date.fromisoformat(valor_data).isoformat() if valor_data else ''
```

A primeira linha lê o campo com alternativa vazia. A segunda usa expressão condicional: só interpreta uma data quando há valor. Se o formato recebido for inválido, a operação lança ValueError, tratado como entrada inadequada pelo método. Vazio é uma ausência aceita, e por isso o filtro de vencidas deve excluir explicitamente data_interna vazia.

Resposta de API na interface

```
const response = await fetch('/api/conta');
const data = await response.json();
if (!response.ok) throw new Error(data.error);
```

fetch devolve uma Promise. await aguarda seu resultado dentro de uma função async sem transformar o servidor em código local. A resposta HTTP e o corpo JSON são objetos diferentes: response contém status e cabeçalhos; data é o corpo interpretado. Uma resposta 403 pode ter JSON perfeitamente válido e ainda representar uma ação recusada. Por isso verificar apenas se json() funcionou é insuficiente.

O api real do projeto acrescenta timeout, verificação de tipo de conteúdo e tratamento de sessão. O run real cuida de estado ocupado e restauração dos controles. Este recorte mostra a lógica central, não deve substituir essas proteções.

Uma fonte substituível para teste

```
class FonteComFalha:
    def buscar(self, tribunal, numero):
        raise FalhaAPI(502, 'Fonte temporariamente indisponível.')
```

Esse objeto oferece a forma buscar esperada pelo serviço. Quando injetado em um teste de consulta, produz erro controlado sem acessar tribunal. Depois da chamada, o teste precisa verificar que o snapshot anterior permanece e que a falha foi registrada. Um teste que apenas espera uma exceção não comprova preservação dos dados.

31 Roteiro de demonstração e autoavaliação

Comece pela apresentação e diga qual problema será mostrado: acompanhar uma mudança e encaminhar um trabalho, com histórico. Abra o treino se a pessoa quiser aprender sem conta. Depois mostre o escritório autenticado e a identificação explícita da demonstração fictícia. Isso evita que alguém interprete o exemplo como consulta real a um processo.

Consulte a primeira referência, simule a novidade e consulte outra vez. Abra a linha do tempo. Mostre o que mudou e registre leitura da versão. Crie uma providência com título concreto e responsáveis. Inicie, descreva evidência e revise. Termine mostrando as contagens e a possibilidade de consultar o histórico depois de novo login.

Para explicar a implementação, abra primeiro a função de evento no frontend, depois a rota de dispatch, depois o método do serviço e por último o teste. Ao apresentar código, use um fluxo com começo e resultado observável; não percorra arquivos em ordem alfabética sem relacioná-los à ação.

Você está pronto para explicar uma parte quando consegue responder sem leitura: qual dado entra; quem pode executar; qual validação impede erro; o que muda no banco; qual resultado a tela recebe; qual teste comprova a regra; qual limitação permanece. Se alguma resposta depender apenas do nome de uma biblioteca, volte ao trecho e reproduza um exemplo menor.

Para a entrevista, descreva o desenvolvimento assistido com precisão e destaque o que você revisou, testou e consegue reproduzir. A confiança técnica vem de demonstrar um comportamento e explicar uma decisão, incluindo suas limitações, sem atribuir ao projeto integrações ou garantias que ele ainda não tem.

Catálogo do código do Caderno Âmbar

Gerado das fontes locais em 4 de novembro de 2026. As linhas são referências desta edição e podem mudar após novas alterações.

Como ler o catálogo

Cada símbolo contém localização, responsabilidade e chamadas diretas extraídas da árvore sintática ou da declaração JavaScript. Chamadas ajudam a navegar; não são prova de que todo ramo é executado em qualquer requisição. O manual principal explica os percursos completos.

backend/acompanhamento.py

Persistência e integração DataJud do caderno pessoal. Sem dados simulados.

FalhaAPI

Linha 18. Classe com base Exception.

FalhaAPI.__init__

Linhas 19 a 23. Assinatura: __init__(self, status, mensagem, *, codigo=None, aguardar=None).

Guarda o status HTTP e inicializa a mensagem da exceção para a camada de resposta.

Chamadas para acompanhar no código: super.__init__, super.

Caderno

Linha 26. Classe com base object.

Caderno.__init__

Linhas 27 a 45. Assinatura: __init__(self, arquivo).

Prepara o diretório e as tabelas originais de usuários, sessões, casos pessoais e consultas.

Chamadas para acompanhar no código: str, Path.parent.mkdir, self.conexao, db.executescript, Path.

Caderno.conexao

Linhas 48 a 56. Assinatura: conexao(self).

Abre SQLite com linhas nomeadas e chaves estrangeiras. O contexto confirma ou desfaz a transação e fecha a conexão.

Chamadas para acompanhar no código: sqlite3.connect, db.execute, db.close.

Caderno.hash_senha

Linhas 59 a 60. Assinatura: hash_senha(senha, salt).

Deriva hash scrypt com salt. O resultado permite validar uma senha sem armazená-la em texto.

Chamadas para acompanhar no código: hashlib.scrypt.hex, hashlib.scrypt, senha.encode, bytes.fromhex.

Caderno.autenticar

Linhas 62 a 85. Assinatura: autenticar(self, dados, cadastro=False).

Valida credenciais, cadastra quando solicitado e cria sessão aleatória com hash e expiração. Login não equivale a e-mail confirmado.

Chamadas para acompanhar no código: dados.get, FalhaAPI, self.conexao, db.execute.fetchone, self.hash_senha, secrets.token_urlsafe, db.execute, isinstance, re.fullmatch, len, secrets.token_hex, hmac.compare_digest, time.time, hashlib.sha256.hexdigest.

Caderno.usuario

Linhas 87 a 93. Assinatura: usuario(self, token).

Resolve um token em usuário somente quando a sessão existe e ainda é válida.

Chamadas para acompanhar no código: self.conexao, db.execute.fetchone, FalhaAPI, db.execute, hashlib.sha256.hexdigest, time.time, hashlib.sha256, token.encode.

Caderno.sair

Linhas 95 a 97. Assinatura: sair(self, token).

Revoga a sessão correspondente ao token. Outras sessões só são eliminadas por uma ação que as revogue explicitamente.

Chamadas para acompanhar no código: self.conexao, db.execute, hashlib.sha256.hexdigest, hashlib.sha256, token.encode.

Caderno.listar

Linhas 99 a 101. Assinatura: listar(self, usuario).

Lista processos no contrato da classe onde está definido. A versão de Equipes é legada e limitada; a interface atual usa Piloto.carteira.

Chamadas para acompanhar no código: self.conexao, dict, db.execute.

Caderno.adicionar

Linhas 103 a 119. Assinatura: adicionar(self, usuario, dados).

Valida entrada e grava processo no escopo autorizado. Na versão de equipes, associa escritório e responsável e recusa duplicação.

Chamadas para acompanhar no código: re.sub, str, dados.get, re.fullmatch, FalhaAPI, self.conexao, isinstance, len, db.execute, apelido.strip.

Caderno.caso

Linhas 121 a 125. Assinatura: caso(self, db, usuario, identificador).

Localiza o processo e verifica o escopo permitido ao usuário. É utilizado por outros métodos antes de trabalhar com IDs recebidos.

Chamadas para acompanhar no código: db.execute.fetchone, FalhaAPI, db.execute.

Caderno.historico

Linhas 127 a 130. Assinatura: historico(self, usuario, identificador).

Devolve tentativas de consulta do processo autorizado, incluindo falhas. Não se confunde com versões do conteúdo nem com eventos de providência.

Chamadas para acompanhar no código: self.conexao, self.caso, dict, db.execute.

Caderno.revisar

Linhas 132 a 138. Assinatura: revisar(self, usuario, identificador).

Registra conferência. Na base antiga altera um marcador; em Equipes registra usuário e versão concreta para preservar a história da leitura.

Chamadas para acompanhar no código: self.conexao, self.caso, db.execute, FalhaAPI.

Caderno.consultar

Linhas 140 a 163. Assinatura: consultar(self, usuario, identificador, cliente=None).

Coordena a consulta e persistência. Em Equipes reserva, obtém fonte, compara e grava; Piloto acrescenta a confirmação de e-mail para fontes reais.

Chamadas para acompanhar no código: json.dumps, self.conexao, dict, db.execute.fetchone, FalhaAPI, BoolOp.buscar, db.execute, self.caso, time.time, DataJud, str.

DataJud

Linha 166. Classe com base object. Consulta pública com paginação documentada e recusa de dados incompletos.

DataJud.tipos_erro

Linhas 169 a 183. Assinatura: tipos_erro(payload).

Lê causas encadeadas do Elasticsearch sem expor motivos ou dados brutos.

Chamadas para acompanhar no código: set, range, isinstance, fila.pop, fila.extend, payload.get, item.get, re.fullmatch, tipos.add.

DataJud.espera

Linhas 186 a 195. Assinatura: espera(resposta, padrao=90).

Auxiliar local do fluxo de acompanhamento. Sua assinatura e as chamadas abaixo indicam os dados necessários; leia o corpo no ponto indicado e relacione-o ao método externo que o chama.

Chamadas para acompanhar no código: str.strip, max, int, str, resposta.headers.get, BinOp.total_seconds, parsedate_to_datetime, datetime.now.

DataJud.corpo

Linhas 198 a 208. Assinatura: corpo(numero, cursor=None).

Auxiliar local do fluxo de acompanhamento. Sua assinatura e as chamadas abaixo indicam os dados necessários; leia o corpo no ponto indicado e relacione-o ao método externo que o chama.

DataJud.buscar

Linhas 210 a 326. Assinatura: buscar(self, tribunal, numero).

Obtém dados segundo o contrato da fonte. DataJud usa HTTP JSON; DataJudValidado acrescenta validação; Demonstracao retorna conteúdo fictício controlado.

Chamadas para acompanhar no código: os.environ.get.strip, time.monotonic, FalhaAPI, os.environ.get, requests.Session, range, registros.sort, set, transporte, resposta.json, payload.get, isinstance, Subscript.get, fonte.get.

backend/aplicacao.py

API FastAPI local: contratos HTTP, sessões e tarefas com persistência.

criar_app

Linhas 31 a 264. Assinatura: criar_app(banco=None, worker=True).

Monta a aplicação FastAPI com um serviço persistente, controles HTTP, rotas e arquivos estáticos. Permite banco temporário e ciclos desativados nos testes.

Chamadas para acompanhar no código: os.environ.get, Piloto, FastAPI, app.add_middleware, app.middleware, app.exception_handler, app.get, app.api_route, app.mount, JSONResponse, secrets.token_hex, request.url.path.startswith, resposta_falha, Response.

criar_app.agendador

Linhas 44 a 48. Assinatura: agendador(acao, intervalo).

Executa uma ação síncrona em thread, registra falhas e aguarda o intervalo sem ocupar o laço assíncrono. É um ciclo local, não uma fila externa.

Chamadas para acompanhar no código: asyncio.sleep, run_in_threadpool, logger.exception.

criar_app.lifespan

Linhas 51 a 57. Assinatura: lifespan(app).

Inicia e cancela os ciclos de consulta, correio e backup durante o tempo de vida da aplicação.

Chamadas para acompanhar no código: task.cancel, asyncio.gather, asyncio.create_task, agendador.

criar_app.limitar

Linhas 64 a 71. Assinatura: limitar(chave, maximo, janela).

Conta tentativas por chave e janela no banco. Recusa excesso com erro 429 antes de executar a operação protegida.

Chamadas para acompanhar no código: caderno.conexao, db.execute, db.execute.fetchone, FalhaAPI, time.time.

criar_app.resposta_falha

Linhas 73 a 79. Assinatura: resposta_falha(exc).

Auxiliar local do fluxo de aplicacao. Sua assinatura e as chamadas abaixo indicam os dados necessários; leia o corpo no ponto indicado e relacione-o ao método externo que o chama.

Chamadas para acompanhar no código: JSONResponse, str.

criar_app.controles

Linhas 82 a 109. Assinatura: controles(request, call_next).

Verifica origem e tamanho da requisição e acrescenta cabeçalhos de segurança, cache e identificação. Não substitui autorização de domínio.

Chamadas para acompanhar no código: app.middleware, secrets.token_hex, request.url.path.startswith, request.url.path.endswith, int, bytearray, request.stream, bytes, call_next, resposta_falha, logger.exception, JSONResponse, request.headers.get, FalhaAPI.

criar_app.falha

Linhas 112 a 112. Assinatura: falha(request, exc).

Converte uma FalhaAPI em resposta JSON com o código HTTP da regra recusada.

Chamadas para acompanhar no código: app.exception_handler, resposta_falha.

criar_app.saude

Linhas 115 a 117. Assinatura: saude().

Executa uma consulta básica ao banco e informa o estado básico do serviço. Não comprova disponibilidade dos fornecedores.

Chamadas para acompanhar no código: app.get, caderno.conexao, db.execute, os.environ.get.

criar_app.runtime

Linhas 120 a 120. Assinatura: runtime().

Entrega a configuração JavaScript do laboratório quando atendido por FastAPI.

Chamadas para acompanhar no código: app.get, Response.

criar_app.carteira

Linhas 123 a 126. Assinatura: carteira(request: Request, org: int | None=None, pagina: int=1, busca: str='', filtro: str='all').

Busca e filtra processos no banco, calcula total e retorna página de vinte itens com resumo da carteira inteira e referência à demonstração.

Chamadas para acompanhar no código: app.get, caderno.usuario, caderno.carteira, request.cookies.get.

criar_app.dados

Linhas 128 a 133. Assinatura: dados(request).

Exige objeto JSON e tipo de conteúdo apropriado antes de encaminhar campos ao serviço.

Chamadas para acompanhar no código: FalhaAPI, isinstance, request.headers.get.split, request.json, request.headers.get.

criar_app.endpoint

Linhas 136 a 139. Assinatura: endpoint(caminho, request: Request).

Recebe as rotas legadas, interpreta corpo e executa dispatch em thread para separar trabalho síncrono do laço HTTP.

Chamadas para acompanhar no código: app.api_route, dados, run_in_threadpool, FalhaAPI.

criar_app.dispatch

Linhas 141 a 261. Assinatura: dispatch(path, request, d).

Escolhe a operação conforme caminho e método, aplica sessão e limites e chama o serviço de domínio. Contém contratos legados ainda não separados em rotas tipadas.

Chamadas para acompanhar no código: request.cookies.get, caderno.usuario, path.split, FalhaAPI, limitar, caderno.autenticar, JSONResponse, r.set_cookie, d.get, consultar_fonte, request.query_params.get, caderno.conta, caderno.solicitar_verificacao, caderno.confirmar_email.

backend/backup.py

Backup consistente e restauração para um NOVO caminho, sem sobrescrever banco.

copiar

Linhas 7 a 18. Assinatura: copiar(origem, destino).

Cria backup consistente SQLite em novo destino e verifica integridade da origem e cópia. Recusa sobrescrever um arquivo existente.

Chamadas para acompanhar no código: destino.exists, destino.parent.mkdir, sqlite3.connect, Path, origem.is_file, ValueError, src.backup, src.close, dst.close, origem.resolve.as_uri, src.execute.fetchone, dst.execute.fetchone, origem.resolve, src.execute.

backend/consulta.py

Requests busca a fonte; Beautiful Soup extrai; Python compara os registros.

ErroConsulta

Linha 16. Classe com base Exception. Falha real de consulta, sem substituição por dados fictícios de sucesso.

validar_entradas

Linhas 20 a 35. Assinatura: validar_entradas(casos: object).

Exige de um a três casos de treino, códigos válidos não repetidos e texto anterior com tamanho limitado.

Chamadas para acompanhar no código: set, ValueError, vistos.add, isinstance, caso.get, len, re.fullmatch.

extrair_e_comparar

Linhas 38 a 58. Assinatura: extrair_e_comparar(html: bytes, casos: list[dict[str, str]]).

Usa Beautiful Soup e seletores conhecidos para extrair movimento/data e comparar com a referência anterior sem inventar registros ausentes.

Chamadas para acompanhar no código: BeautifulSoup, validar_entradas, pagina.select_one, resultados.append, registro.select_one, movimento.get_text, data.get_text.

consultar_fonte

Linhas 61 a 84. Assinatura: consultar_fonte(url: str, casos: list[dict[str, str]], timeout=(3, 10)).

Obtém HTML por Requests com timeout, valida HTTP e chama a extração; inclui evidência de ferramenta e hash da resposta no relatório.

Chamadas para acompanhar no código: validar_entradas, extrair_e_comparar, datetime.now.isoformat, hashlib.sha256.hexdigest, requests.Session, sessao.get, resposta.raise_for_status, ErroConsulta, datetime.now, hashlib.sha256.

main

Linhas 87 a 102. Assinatura: main().

Ponto de entrada de terminal do módulo. Interpreta argumentos ou inicia o serviço específico; confira o arquivo antes de executar.

Chamadas para acompanhar no código: argparse.ArgumentParser, parser.add_argument, parser.parse_args, args.saida.parent.mkdir, args.saida.write_text, print, any, consultar_fonte, json.dumps, parser.exit, Path.

backend/correio.py

Caixa local e SMTP com TLS; aceite pelo servidor não significa entrega final.

entregar

Linhas 12 a 27. Assinatura: entregar(pasta, destinatario, assunto, conteudo, chave=None).

Em desenvolvimento grava mensagem local. Em produção envia por SMTP com STARTTLS e distingue aceite de entrega confirmada.

Chamadas para acompanhar no código: EmailMessage, msg.set_content, os.environ.get, pasta.mkdir, BinOp.write_text, all, FalhaAPI, Constant.join, Path, json.dumps, smtplib.SMTP, smtp.starttls, smtp.login, smtp.send_message.

backend/equipes.py

Migração aditiva, equipes, versões, tarefas e auditoria do Caderno Âmbar.

Equipes

Linha 15. Classe com base Caderno.

Equipes.__init__

Linhas 16 a 52. Assinatura: __init__(self, arquivo).

Executa a base pessoal, acrescenta tabelas e colunas de escritório e associa contas antigas sem apagar dados.

Chamadas para acompanhar no código: super.__init__, self.conexao, db.executescript, db.execute, db.execute.fetchall, super, self._org, time.time.

Equipes.conexao

Linhas 54 a 56. Assinatura: conexao(self).

Abre SQLite com linhas nomeadas e chaves estrangeiras. O contexto confirma ou desfaz a transação e fecha a conexão.

Chamadas para acompanhar no código: super.conexao, super.

Equipes._org

Linhas 58 a 64. Assinatura: _org(self, db, usuario).

Encontra o primeiro escritório de uma conta ou cria o espaço pessoal e associa casos antigos ainda sem organização.

Chamadas para acompanhar no código: db.execute.fetchone, db.execute.

Equipes.org

Linhas 66 a 70. Assinatura: org(self, db, usuario, org=None, admin=False).

Verifica se o usuário pertence ao escritório e, quando solicitado, se é administrador. Não aceita um ID como prova de acesso.

Chamadas para acompanhar no código: db.execute.fetchone, int, self._org, FalhaAPI, db.execute.

Equipes.audit

Linhas 72 a 73. Assinatura: audit(self, db, org, usuario, acao, caso=None).

Grava ação, escritório, pessoa e instante para rastreabilidade operacional.

Chamadas para acompanhar no código: db.execute, time.time.

Equipes.contexto

Linhas 75 a 78. Assinatura: contexto(self, usuario).

Lista escritórios e papéis da pessoa autenticada para seleção da área ativa.

Chamadas para acompanhar no código: self.conexao, self._org, dict, db.execute.

Equipes.listar

Linhas 80 a 86. Assinatura: listar(self, usuario, org=None).

Lista processos no contrato da classe onde está definido. A versão de Equipes é legada e limitada; a interface atual usa Piloto.carteira.

Chamadas para acompanhar no código: self.conexao, self.org, dict, db.execute.

Equipes.caso

Linhas 88 a 92. Assinatura: caso(self, db, usuario, identificador).

Localiza o processo e verifica o escopo permitido ao usuário. É utilizado por outros métodos antes de trabalhar com IDs recebidos.

Chamadas para acompanhar no código: db.execute.fetchone, self.org, FalhaAPI, db.execute.

Equipes.adicionar

Linhas 94 a 105. Assinatura: adicionar(self, usuario, dados).

Valida entrada e grava processo no escopo autorizado. Na versão de equipes, associa escritório e responsável e recusa duplicação.

Chamadas para acompanhar no código: validar_numero, str.strip, dados.get, len, FalhaAPI, self.conexao, db.execute, self.org, db.execute.fetchone, self.audit, str.

Equipes.alterar

Linhas 107 a 122. Assinatura: alterar(self, usuario, id, dados).

Valida alterações organizacionais de processo e participante, atualiza arquivamento e periodicidade e ajusta sua tarefa agendada.

Chamadas para acompanhar no código: self.conexao, self.caso, int, db.execute, self.audit, dados.get, db.execute.fetchone, FalhaAPI, isinstance, len, bool, time.time.

Equipes.demonstracao

Linhas 124 a 131. Assinatura: demonstracao(self, usuario, org=None).

Retorna ou cria um processo explicitamente fictício para aprender o fluxo persistente sem consulta a tribunal.

Chamadas para acompanhar no código: self.conexao, self.org, db.execute.fetchone, self.audit, db.execute.

Equipes.agendar_consulta

Linhas 133 a 153. Assinatura: agendar_consulta(self, usuario, id).

Autoriza e persiste uma solicitação de atualização sem acessar o CNJ na mesma requisição. Preserva a espera existente, evita pedidos duplicados e registra autoria; Piloto exige confirmação de e-mail.

Chamadas para acompanhar no código: time.time, self.conexao, db.execute, self.caso, db.execute.fetchone, max, self.audit, FalhaAPI, os.environ.get.

Equipes.consultar

Linhas 155 a 225. Assinatura: consultar(self, usuario, id, cliente=None).

Coordena a consulta e persistência. Em Equipes reserva, obtém fonte, compara e grava; Piloto acrescenta a confirmação de e-mail para fontes reais.

Chamadas para acompanhar no código: time.time, self.conexao, db.execute, dict, db.execute.fetchone, normalizar, self.caso, FalhaAPI, max, BoolOp.buscar, diferencas, self.audit, isinstance, os.environ.get.

Equipes.revisar

Linhas 227 a 234. Assinatura: revisar(self, usuario, id, versao_id, observacao='').

Registra conferência. Na base antiga altera um marcador; em Equipes registra usuário e versão concreta para preservar a história da leitura.

Chamadas para acompanhar no código: FalhaAPI, self.conexao, self.caso, db.execute, self.audit, isinstance, len, db.execute.fetchone, time.time.

Equipes.detalhe

Linhas 236 a 240. Assinatura: detalhe(self, usuario, id).

Reúne versões e conferências do processo autorizado para construir sua linha do tempo.

Chamadas para acompanhar no código: self.conexao, self.caso, dict, db.execute.

Equipes.membros

Linhas 242 a 245. Assinatura: membros(self, usuario, org).

Lista participantes e papéis de um escritório autorizado. Não fornece hashes de senha ou tokens.

Chamadas para acompanhar no código: self.conexao, self.org, dict, db.execute.

Equipes.convidar

Linhas 247 a 254. Assinatura: convidar(self, usuario, org, email).

Cria token temporário destinado a um e-mail e escritório. Na classe Piloto, exige confirmação do e-mail do administrador antes de delegar à base.

Chamadas para acompanhar no código: secrets.token_urlsafe, FalhaAPI, self.conexao, self.org, db.execute, self.audit, isinstance, len, hashlib.sha256.hexdigest, email.lower, time.time, hashlib.sha256, token.encode.

Equipes.aceitar

Linhas 256 a 265. Assinatura: aceitar(self, usuario, token).

Consome um convite válido para a conta de destino e cria a associação. Na classe Piloto, exige e-mail confirmado antes da operação.

Chamadas para acompanhar no código: self.conexao, db.execute, db.execute.fetchone, self.audit, FalhaAPI, hashlib.sha256.hexdigest, time.time, hashlib.sha256, token.encode.

Equipes.revogar

Linhas 267 a 274. Assinatura: revogar(self, usuario, org, alvo).

Remove associação e reatribui responsabilidades na implementação atual de Escritorio. A implementação de Equipes é anterior à rotina.

Chamadas para acompanhar no código: self.conexao, self.org, db.execute, self.audit, int, FalhaAPI.

Equipes.importar

Linhas 276 a 294. Assinatura: importar(self, usuario, org, conteudo, confirmar=False).

Lê e valida CSV, detecta duplicação e produz prévia ou processa as linhas válidas quando confirmado. O resultado é individual por linha.

Chamadas para acompanhar no código: list, enumerate, FalhaAPI, csv.DictReader, len, self.conexao, self.org, resultados.append, isinstance, io.StringIO, validar_numero, vistos.add, db.execute, d.get.

Equipes.tick

Linhas 296 a 304. Assinatura: tick(self).

Ciclo legado que reúne processamento. A aplicação atual utiliza ciclos separados para reduzir dependência entre consulta, e-mail e backup.

Chamadas para acompanhar no código: self.email_tick, self.backup_tick, self.conexao, dict, self.consultar, db.execute, time.time.

Equipes.backup_tick

Linhas 306 a 314. Assinatura: backup_tick(self).

Quando habilitado, prepara uma cópia consistente diária no diretório configurado. Não implementa armazenamento externo criptografado nem retenção automática.

Chamadas para acompanhar no código: Path, pasta.mkdir, os.environ.get, destino.exists, copiar, str, datetime.now.strftime, datetime.now.

Equipes.email_tick

Linhas 316 a 329. Assinatura: email_tick(self).

Processa avisos elegíveis. A implementação atual reserva e revalida em transações curtas e envia fora delas; a implementação de Equipes é legada.

Chamadas para acompanhar no código: pasta.mkdir, self.conexao, db.execute, db.execute.fetchall, __import__.Path, entregar, __import__, time.time.

backend/fontes.py

Contratos, validação CNJ e comparação independente de HTTP/persistência.

validar_numero

Linhas 11 a 20. Assinatura: validar_numero(valor, tribunal).

Remove pontuação, valida vinte dígitos, segmento/tribunal suportado e dígitos verificadores do CNJ. Não comprova existência do processo.

Chamadas para acompanhar no código: re.sub, str, FalhaAPI, int, re.fullmatch.

serializar

Linhas 23 a 24. Assinatura: serializar(dado).

Produz JSON estável para comparar conteúdo sem depender da ordem das chaves ou dos espaços.

Chamadas para acompanhar no código: json.dumps.

versao

Linhas 27 a 28. Assinatura: versao(dado).

Calcula SHA-256 do conteúdo serializado de forma estável para ajudar a identificar igualdade.

Chamadas para acompanhar no código: hashlib.sha256.hexdigest, hashlib.sha256, serializar.encode, serializar.

Fonte

Linha 31. Classe com base Protocol.

Fonte.buscar

Linhas 32 a 32. Assinatura: buscar(self, tribunal: str, numero: str).

Obtém dados segundo o contrato da fonte. DataJud usa HTTP JSON; DataJudValidado acrescenta validação; Demonstracao retorna conteúdo fictício controlado.

normalizar

Linhas 35 a 48. Assinatura: normalizar(dado).

Verifica o contrato da fonte, ordena registros e elimina movimentos exatamente repetidos. Não interpreta o mérito jurídico.

Chamadas para acompanhar no código: FalhaAPI, registros.append, sorted, isinstance, dado.get, serializar, registro.get, m.get, movimentos.values.

diferencas

Linhas 51 a 63. Assinatura: diferencas(anterior, atual).

Compara movimentos por grau e dados cadastrais para classificar primeiro registro, igualdade, inclusão, remoção/correção ou cadastro alterado.

Chamadas para acompanhar no código: sorted, movimentos, cadastro, serializar, r.get, depois.keys, antes.keys, r.items.

diferencas.movimentos

Linhas 54 a 56. Assinatura: movimentos(d).

Transforma movimentos em mapa com identificação estável que inclui o grau, para comparação entre conjuntos.

Chamadas para acompanhar no código: serializar, r.get.

diferencas.cadastro

Linhas 57 a 58. Assinatura: cadastro(d).

Separa os dados cadastrais de cada registro dos movimentos para comparar essas duas categorias independentemente.

Chamadas para acompanhar no código: sorted, serializar, r.items.

DataJudValidado

Linha 66. Classe com base DataJud.

DataJudValidado.buscar

Linhas 67 a 69. Assinatura: buscar(self, tribunal, numero).

Obtém dados segundo o contrato da fonte. DataJud usa HTTP JSON; DataJudValidado acrescenta validação; Demonstracao retorna conteúdo fictício controlado.

Chamadas para acompanhar no código: validar_numero, normalizar, super.buscar, super.

Demonstracao

Linha 72. Classe com base object.

Demonstracao.__init__

Linhas 73 a 74. Assinatura: __init__(self, etapa=0).

Guarda a etapa fictícia que determina os movimentos retornados pelo exemplo.

Demonstracao.buscar

Linhas 76 a 81. Assinatura: buscar(self, tribunal, numero).

Obtém dados segundo o contrato da fonte. DataJud usa HTTP JSON; DataJudValidado acrescenta validação; Demonstracao retorna conteúdo fictício controlado.

Chamadas para acompanhar no código: movimentos.append.

backend/iniciar_datajud.py

Obtém a chave pública publicada pelo CNJ e inicia o servidor local.

backend/piloto.py

Preparação do piloto: migração preservadora, carteira paginada e e-mail verificado.

Este serviço estende a rotina existente. Não consulta fontes por conta própria e não transforma a demonstração em prova de integração com um tribunal.

Piloto

Linha 21. Classe com base Escritorio.

Piloto.__init__

Linhas 22 a 38. Assinatura: __init__(self, arquivo).

Executa a rotina, migra a unicidade da carteira e cria confirmação de e-mail e solicitações administrativas de dados.

Chamadas para acompanhar no código: super.__init__, self.migrar_carteira, self.conexao, db.executescript, db.execute, super, time.time.

Piloto.migrar_carteira

Linhas 40 a 68. Assinatura: migrar_carteira(self).

Faz backup, reconstrói casos com unicidade por escritório, conserva colunas e IDs e verifica vínculos antes de confirmar. Usa rollback em falha.

Chamadas para acompanhar no código: Path.with_name, copiar, sqlite3.connect, self.conexao, re.search, db.execute, re.sub, Constant.join, db.execute.fetchone, db.commit, db.close, Path, RuntimeError, db.rollback.

Piloto.conta

Linhas 70 a 73. Assinatura: conta(self, usuario).

Fornece ID, e-mail e instante de verificação da própria conta. Não inclui dados de sessão ou senha.

Chamadas para acompanhar no código: self.conexao, db.execute.fetchone, dict, db.execute.

Piloto.exigir_email

Linhas 75 a 77. Assinatura: exigir_email(self, usuario).

Recusa operações protegidas enquanto a conta não possuir registro de confirmação.

Chamadas para acompanhar no código: FalhaAPI, self.conta.

Piloto.solicitar_verificacao

Linhas 79 a 96. Assinatura: solicitar_verificacao(self, usuario).

Cria segredo aleatório de confirmação, persiste somente hash/expiração e entrega pelo canal configurado. Remove o token se o envio falhar.

Chamadas para acompanhar no código: self.conta, secrets.token_urlsafe, hashlib.sha256.hexdigest, self.conexao, db.execute, entregar, hashlib.sha256, segredo.encode, Path, time.time.

Piloto.confirmar_email

Linhas 98 a 108. Assinatura: confirmar_email(self, usuario, token).

Verifica conta, hash e validade dentro da transação, registra confirmação e consome os tokens. O mesmo código não pode ser reutilizado.

Chamadas para acompanhar no código: hashlib.sha256.hexdigest, self.conexao, db.execute, db.execute.fetchone, hashlib.sha256, FalhaAPI, str.encode, time.time, str.

Piloto.convidar

Linhas 110 a 112. Assinatura: convidar(self, usuario, org, email).

Cria token temporário destinado a um e-mail e escritório. Na classe Piloto, exige confirmação do e-mail do administrador antes de delegar à base.

Chamadas para acompanhar no código: self.exigir_email, super.convidar, super.

Piloto.aceitar

Linhas 114 a 116. Assinatura: aceitar(self, usuario, token).

Consome um convite válido para a conta de destino e cria a associação. Na classe Piloto, exige e-mail confirmado antes da operação.

Chamadas para acompanhar no código: self.exigir_email, super.aceitar, super.

Piloto.agendar_consulta

Linhas 118 a 120. Assinatura: agendar_consulta(self, usuario, id).

Autoriza e persiste uma solicitação de atualização sem acessar o CNJ na mesma requisição. Preserva a espera existente, evita pedidos duplicados e registra autoria; Piloto exige confirmação de e-mail.

Chamadas para acompanhar no código: self.exigir_email, super.agendar_consulta, super.

Piloto.consultar

Linhas 122 a 127. Assinatura: consultar(self, usuario, id, cliente=None).

Coordena a consulta e persistência. Em Equipes reserva, obtém fonte, compara e grava; Piloto acrescenta a confirmação de e-mail para fontes reais.

Chamadas para acompanhar no código: super.consultar, self.conexao, self.caso, self.exigir_email, super.

Piloto.carteira

Linhas 129 a 166. Assinatura: carteira(self, usuario, org=None, pagina=1, busca='', filtro='all').

Busca e filtra processos no banco, calcula total e retorna página de vinte itens com resumo da carteira inteira e referência à demonstração.

Chamadas para acompanhar no código: max, FalhaAPI, int, str.strip, self.conexao, self.org, min, dict, db.execute.fetchone, where.append, args.extend, Constant.join, str, db.execute.

Piloto.resumo_diario

Linhas 168 a 175. Assinatura: resumo_diario(self, usuario, org).

Conta as providências atribuídas, vencidas e aguardando revisão do usuário no escritório autorizado.

Chamadas para acompanhar no código: self.conexao, self.org, dict, db.execute.fetchone, db.execute, date.today.isoformat, date.today.

Piloto.solicitar_dados

Linhas 177 a 185. Assinatura: solicitar_dados(self, usuario, org, tipo, motivo).

Registra solicitação administrativa de retenção ou exclusão com justificativa. Não executa apagamento automático.

Chamadas para acompanhar no código: FalhaAPI, self.conexao, self.org, self.audit, isinstance, db.execute, len, motivo.strip, time.time.

Piloto.exportar_escritorio

Linhas 187 a 200. Assinatura: exportar_escritorio(self, usuario, org).

Exporta dados relacionados do escritório para administração, omitindo credenciais, sessões e tokens. Não é um restaurador.

Chamadas para acompanhar no código: self.conexao, self.org, self.audit, time.time, dict, db.execute.

backend/rotina.py

Providências jurídicas e processamento de fila sem rede em transações SQLite.

Escritorio

Linha 23. Classe com base Equipes.

Escritorio.__init__

Linhas 24 a 45. Assinatura: __init__(self, arquivo).

Executa a base de equipes e cria providências, seus eventos e campos de reserva da fila de e-mail.

Chamadas para acompanhar no código: super.__init__, self.conexao, db.executescript, db.execute, super, time.time.

Escritorio.me

Linhas 47 a 51. Assinatura: me(self, usuario, org).

Devolve identidade mínima e escritório autorizado para a rotina, sem fornecer dados de autenticação.

Chamadas para acompanhar no código: self.conexao, self.org, db.execute.fetchone, db.execute.

Escritorio.carteira_exportavel

Linhas 53 a 58. Assinatura: carteira_exportavel(self, usuario, org).

Consulta metadados de toda a carteira para CSV, sem depender da página visível nem carregar os snapshots completos.

Chamadas para acompanhar no código: self.conexao, self.org, dict, db.execute.

Escritorio.participante

Linhas 60 a 62. Assinatura: participante(self, db, org, usuario).

Confirma que executor e revisor pertencem ao escritório da providência antes de aceitar a atribuição.

Chamadas para acompanhar no código: db.execute.fetchone, FalhaAPI, db.execute.

Escritorio.texto

Linhas 65 a 69. Assinatura: texto(dados, campo, minimo, maximo).

Valida se um campo é texto e respeita limites de tamanho após remover espaços laterais.

Chamadas para acompanhar no código: dados.get, valor.strip, FalhaAPI, isinstance, len.

Escritorio.criar_providencia

Linhas 71 a 95. Assinatura: criar_providencia(self, usuario, dados).

Valida processo, versão, participantes, prioridade e data interna opcional e grava a tarefa e seu primeiro evento.

Chamadas para acompanhar no código: self.texto, dados.get, int, FalhaAPI, self.conexao, db.execute, self.caso, self.participante, time.time, self.audit, date.fromisoformat.isoformat, db.execute.fetchone, json.dumps, date.fromisoformat.

Escritorio.rotina

Linhas 97 a 118. Assinatura: rotina(self, usuario, org, filtro='minhas', pagina=1, busca='').

Aplica filtros, busca e paginação SQL às providências e calcula contagens por responsabilidade e revisão.

Chamadas para acompanhar no código: max, FalhaAPI, int, str, self.conexao, self.org, dict, args.append, args.extend, Constant.join, db.execute.fetchone, date.today.isoformat, db.execute, date.today.

Escritorio.providencia

Linhas 120 a 124. Assinatura: providencia(self, db, usuario, id).

Carrega a atividade e verifica o acesso ao escritório associado antes de devolver o registro.

Chamadas para acompanhar no código: db.execute.fetchone, self.org, FalhaAPI, db.execute.

Escritorio.alterar_providencia

Linhas 126 a 149. Assinatura: alterar_providencia(self, usuario, id, dados).

Aplica permissão, revisão otimista e transições. Exige evidência para revisão/conclusão e comentário para reabertura.

Chamadas para acompanhar no código: self.conexao, db.execute, self.providencia, dados.get, self.texto, time.time, self.audit, FalhaAPI, db.execute.fetchone, isinstance, len, TRANSICOES.get, evidencia.strip, json.dumps.

Escritorio.historico_providencia

Linhas 151 a 155. Assinatura: historico_providencia(self, usuario, id).

Retorna eventos da atividade autorizada, com identificação do participante e ordem histórica.

Chamadas para acompanhar no código: self.conexao, self.providencia, dict, db.execute.

Escritorio.consultas_tick

Linhas 157 a 171. Assinatura: consultas_tick(self).

Seleciona até três consultas vencidas e executa com paralelismo limitado. Cada consulta conserva sua reserva individual.

Chamadas para acompanhar no código: self.conexao, dict, self.consultar, db.execute, time.time.

Escritorio.email_tick

Linhas 173 a 200. Assinatura: email_tick(self).

Processa avisos elegíveis. A implementação atual reserva e revalida em transações curtas e envia fora delas; a implementação de Equipes é legada.

Chamadas para acompanhar no código: time.time, range, secrets.token_hex, self.conexao, db.execute, db.execute.fetchone, entregar, Path, min.

Escritorio.operacao

Linhas 202 a 208. Assinatura: operacao(self, usuario, org).

Entrega diagnóstico administrativo das consultas e mensagens recentes do escritório. Não é monitoramento externo de toda a infraestrutura.

Chamadas para acompanhar no código: self.conexao, self.org, dict, db.execute.

Escritorio.revogar

Linhas 210 a 227. Assinatura: revogar(self, usuario, org, alvo).

Remove associação e reatribui responsabilidades na implementação atual de Escritorio. A implementação de Equipes é anterior à rotina.

Chamadas para acompanhar no código: self.conexao, db.execute, self.org, db.execute.fetchall, self.audit, int, FalhaAPI, time.time, json.dumps.

Escritorio.reprocessar

Linhas 229 a 246. Assinatura: reprocessar(self, usuario, org, tipo, id).

Verifica administração e estado da fila antes de liberar uma nova tentativa; não confirma que a fonte ou o provedor voltou a funcionar.

Chamadas para acompanhar no código: self.conexao, self.org, self.audit, self.caso, db.execute.fetchone, db.execute, FalhaAPI, time.time.

backend/rpa_navegador.py

Playwright opera o Caderno Âmbar e verifica a consulta Python de ponta a ponta.

executar

Linhas 17 a 108. Assinatura: executar(visivel=False, diretorio=Path('output/rpa')).

Executa uma operação no contexto do chamador. No worker, chama a consulta elegível; no roteiro Playwright, conduz o percurso de navegador.

Chamadas para acompanhar no código: Path, diretorio.mkdir, criar_servidor, threading.Thread, thread.start, print, datetime.now.isoformat, version, etapas.append, servidor.shutdown, servidor.server_close, thread.join, BinOp.write_text, sync_playwright.

executar.passo

Linhas 28 a 30. Assinatura: passo(texto).

Registra uma etapa de execução no relatório do roteiro de navegador.

Chamadas para acompanhar no código: etapas.append, print, len.

backend/servidor.py

Servidor local que conecta a interface à consulta Python.

ServidorAmbar

Linha 21. Classe com base ThreadingHTTPServer.

Handler

Linha 26. Classe com base SimpleHTTPRequestHandler.

Handler.token

Linhas 27 a 30. Assinatura: token(self).

Lê o cookie de sessão no servidor legado, sem validar a autorização por si só.

Chamadas para acompanhar no código: SimpleCookie, cookie.load, self.headers.get.

Handler.acompanhamento

Linhas 32 a 65. Assinatura: acompanhamento(self).

Despacha as operações do caderno pessoal no servidor legado, anterior ao contrato atual de equipes.

Chamadas para acompanhar no código: caderno.usuario, re.fullmatch, FalhaAPI, urlsplit, self.headers.get, int, json.loads, caderno.autenticar, self.token, caderno.sair, self.rfile.read, isinstance, path.endswith, caderno.listar.

Handler.responder_acompanhamento

Linhas 67 a 73. Assinatura: responder_acompanhamento(self).

Adapta erros e respostas do serviço pessoal para HTTP no servidor legado.

Chamadas para acompanhar no código: self.json_response, self.acompanhamento, str.

Handler.json_response

Linhas 75 a 85. Assinatura: json_response(self, status, data).

Escreve status, cabeçalhos e corpo JSON no protocolo HTTP do servidor legado.

Chamadas para acompanhar no código: json.dumps.encode, self.send_response, self.send_header, getattr, self.end_headers, self.wfile.write, str, json.dumps, len.

Handler.do_GET

Linhas 87 a 102. Assinatura: do_GET(self).

Trata uma requisição GET no servidor legado, servindo configuração, API pessoal ou arquivo conforme o caminho.

Chamadas para acompanhar no código: path.startswith, super.do_GET, urlsplit, self.responder_acompanhamento, self.json_response, self.send_response, self.send_header, self.end_headers, self.wfile.write, str, super, len.

Handler.do_POST

Linhas 104 a 138. Assinatura: do_POST(self).

Lê e valida a ação POST no laboratório legado e responde com resultado ou erro. A versão FastAPI usa outro caminho.

Chamadas para acompanhar no código: urlsplit.path.startswith, print, self.json_response, self.responder_acompanhamento, int, json.loads, validar_entradas, consultar_fonte, urlsplit, self.headers.get, ValueError, self.rfile.read, audit_file.parent.mkdir, self.headers.get.split.

criar_servidor

Linhas 141 a 144. Assinatura: criar_servidor(porta=8092, banco=None).

Cria a instância HTTP do laboratório legado. Não monta a aplicação FastAPI completa de equipes.

Chamadas para acompanhar no código: ServidorAmbar, Caderno, partial, os.environ.get, str.

main

Linhas 147 a 157. Assinatura: main().

Ponto de entrada de terminal do módulo. Interpreta argumentos ou inicia o serviço específico; confira o arquivo antes de executar.

Chamadas para acompanhar no código: argparse.ArgumentParser, parser.add_argument, parser.parse_args, criar_servidor, print, server.serve_forever.

backend/test_acompanhamento.py

Módulo de testes de comportamento do projeto.

IntegracaoTests

Linha 11. Classe com base unittest.TestCase.

IntegracaoTests.setUp

Linhas 12 a 21. Assinatura: setUp(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: pasta.mkdir, tempfile.TemporaryDirectory, criar_servidor, threading.Thread, self.thread.start, requests.Session, Path, Path.resolve.

IntegracaoTests.tearDown

Linhas 23 a 28. Assinatura: tearDown(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: self.http.close, self.server.shutdown, self.server.server_close, self.thread.join, self.tmp.cleanup.

IntegracaoTests.post

Linhas 30 a 31. Assinatura: post(self, path, dados).

Auxiliar de teste que envia JSON para a API local com uma sessão Requests; não altera o transporte da aplicação.

Chamadas para acompanhar no código: self.http.post.

IntegracaoTests.test_fluxo_persistente_e_isolamento

Linhas 33 a 52. Assinatura: test_fluxo_persistente_e_isolamento(self).

Cenário automatizado: fluxo persistente e isolamento. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, Caderno, reaberto.usuario, self.post, self.post.json, patch.object, len, self.http.get, reaberto.historico, reaberto.listar.

IntegracaoTests.test_comparacao_e_falha_preservam_snapshot

Linhas 54 a 73. Assinatura: test_comparacao_e_falha_preservam_snapshot(self).

Cenário automatizado: comparacao e falha preservam snapshot. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: c.usuario, Mock, self.assertEqual, liberar, FalhaAPI, c.autenticar, c.adicionar, self.assertRaises, c.consultar, c.conexao, db.execute, c.listar, c.historico.

IntegracaoTests.test_comparacao_e_falha_preservam_snapshot.liberar

Linhas 61 a 62. Assinatura: liberar().

Libera uma sincronização do cenário concorrente de teste para permitir que a consulta simulada prossiga.

Chamadas para acompanhar no código: c.conexao, db.execute.

IntegracaoTests.test_datajud_configuracao_e_resposta

Linhas 75 a 83. Assinatura: test_datajud_configuracao_e_resposta(self).

Cenário automatizado: datajud configuracao e resposta. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: Mock, patch.dict, patch, self.assertEqual, self.assertTrue, self.assertRaises, DataJud.buscar, Subscript.endswith, DataJud.

IntegracaoTests.test_datajud_conexao_direta_e_limite_temporario

Linhas 85 a 93. Assinatura: test_datajud_conexao_direta_e_limite_temporario(self).

Cenário automatizado: datajud conexao direta e limite temporario. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: Mock, self.assertEqual, self.assertIn, self.assertIs, sessao.close.assert_called_once, patch.dict, patch, str, self.assertRaises, DataJud.buscar, DataJud.

IntegracaoTests.test_origem_e_senha

Linhas 95 a 97. Assinatura: test_origem_e_senha(self).

Cenário automatizado: origem e senha. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, self.http.post, self.post.

backend/test_api_v2.py

Módulo de testes de comportamento do projeto.

ApiV2Tests

Linha 12. Classe com base unittest.TestCase.

ApiV2Tests.setUpClass

Linhas 14 a 25. Assinatura: setUpClass(cls).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: tempfile.TemporaryDirectory, criar_app, uvicorn.Server, threading.Thread, cls.th.start, range, socket.socket, sock.bind, uvicorn.Config, time.sleep, sock.getsockname, Path, Path.resolve.

ApiV2Tests.tearDownClass

Linhas 28 a 29. Assinatura: tearDownClass(cls).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: cls.th.join, cls.tmp.cleanup.

ApiV2Tests.setUp

Linhas 31 a 32. Assinatura: setUp(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: requests.Session.

ApiV2Tests.tearDown

Linhas 33 a 33. Assinatura: tearDown(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: self.s.close.

ApiV2Tests.post

Linhas 34 a 34. Assinatura: post(self, path, data).

Auxiliar de teste que envia JSON para a API local com uma sessão Requests; não altera o transporte da aplicação.

Chamadas para acompanhar no código: self.s.post.

ApiV2Tests.test_fluxo_http_completo

Linhas 36 a 54. Assinatura: test_fluxo_http_completo(self).

Cenário automatizado: fluxo http completo. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.s.get, self.assertEqual, self.s.get.json, self.post, self.post.json, self.assertFalse, self.assertTrue, pagina.headers.get, len.

ApiV2Tests.test_origem_json_e_limites

Linhas 56 a 60. Assinatura: test_origem_json_e_limites(self).

Cenário automatizado: origem json e limites. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.s.post, self.assertEqual.

ApiV2Tests.test_percurso_publico_separado

Linhas 62 a 68. Assinatura: test_percurso_publico_separado(self).

Cenário automatizado: percurso publico separado. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertNotIn, self.assertIn, self.s.get, self.assertEqual.

ApiV2Tests.test_recuperacao_uso_unico

Linhas 70 a 82. Assinatura: test_recuperacao_uso_unico(self).

Cenário automatizado: recuperacao uso unico. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.post, self.assertEqual, next, self.app.state.caderno.conexao, json.loads, p.read_text, BinOp.glob, self.s.get, db.execute.fetchone, db.execute, Path, m.get.

ApiV2Tests.test_rotina_http

Linhas 84 a 100. Assinatura: test_rotina_http(self).

Cenário automatizado: rotina http. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.post, self.assertEqual, self.s.get.json, self.post.json, r.json, len, self.s.get.

ApiV2Tests.test_piloto_carteira_conta_e_dados_autorizados

Linhas 102 a 127. Assinatura: test_piloto_carteira_conta_e_dados_autorizados(self).

Cenário automatizado: piloto carteira conta e dados autorizados. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, self.post, self.assertIsNone, next, self.s.get, self.assertIn, self.assertNotIn, json.loads, exportacao.json, requests.Session, outra.post, self.s.get.json, p.read_text, BinOp.glob.

ApiV2Tests.test_agendamento_http_aceita_sem_chamar_fonte

Linhas 129 a 143. Assinatura: test_agendamento_http_aceita_sem_chamar_fonte(self).

Cenário automatizado: agendamento http aceita sem chamar fonte. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.post, self.assertEqual, self.s.get.json, self.app.state.caderno.conexao, db.execute, self.post.json, patch.dict, patch, range, fonte.assert_not_called, self.assertTrue, next, self.s.get, r.json.

ApiV2Tests.test_falha_da_fonte_devolve_codigo_espera_e_historico

Linhas 145 a 170. Assinatura: test_falha_da_fonte_devolve_codigo_espera_e_historico(self).

Cenário automatizado: falha da fonte devolve codigo espera e historico. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.post, FalhaAPI, self.s.get.json, self.assertEqual, self.app.state.caderno.conexao, db.execute, self.post.json, patch.dict, patch, len, self.s.get, time.time, r.json, numero.

backend/test_consulta.py

Verifica comparação, falhas de fonte e contrato da API com HTTP real local.

ConsultaTests

Linha 12. Classe com base unittest.TestCase.

ConsultaTests.setUpClass

Linhas 14 a 20. Assinatura: setUpClass(cls).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: criar_servidor, threading.Thread, cls.thread.start, requests.Session.

ConsultaTests.tearDownClass

Linhas 23 a 27. Assinatura: tearDownClass(cls).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: cls.session.close, cls.server.shutdown, cls.server.server_close, cls.thread.join.

ConsultaTests.test_http_extracao_e_comparacao

Linhas 29 a 38. Assinatura: test_http_extracao_e_comparacao(self).

Cenário automatizado: http extracao e comparacao. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: consultar_fonte, self.assertEqual, self.assertTrue.

ConsultaTests.test_reconsulta_sem_mudanca

Linhas 40 a 43. Assinatura: test_reconsulta_sem_mudanca(self).

Cenário automatizado: reconsulta sem mudanca. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: consultar_fonte, self.assertFalse.

ConsultaTests.test_404_nao_gera_sucesso

Linhas 45 a 47. Assinatura: test_404_nao_gera_sucesso(self).

Cenário automatizado: 404 nao gera sucesso. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertRaises, consultar_fonte.

ConsultaTests.test_registro_ausente_nao_inventa_dado

Linhas 49 a 52. Assinatura: test_registro_ausente_nao_inventa_dado(self).

Cenário automatizado: registro ausente nao inventa dado. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: extrair_e_comparar, self.assertFalse, self.assertNotIn.

ConsultaTests.test_entradas_invalidas

Linhas 54 a 58. Assinatura: test_entradas_invalidas(self).

Cenário automatizado: entradas invalidas. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.subTest, self.assertRaises, validar_entradas.

ConsultaTests.test_api_e_rejeicao_de_json_invalido

Linhas 60 a 67. Assinatura: test_api_e_rejeicao_de_json_invalido(self).

Cenário automatizado: api e rejeicao de json invalido. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.session.post, self.assertEqual, resposta.json.

ConsultaTests.test_configuracao_ativa_python

Linhas 69 a 71. Assinatura: test_configuracao_ativa_python(self).

Cenário automatizado: configuracao ativa python. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.session.get, self.assertIn.

backend/test_correio.py

Módulo de testes de comportamento do projeto.

CorreioTests

Linha 8. Classe com base unittest.TestCase.

CorreioTests.test_producao_sem_configuracao

Linhas 9 a 11. Assinatura: test_producao_sem_configuracao(self).

Cenário automatizado: producao sem configuracao. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: patch.dict, self.assertRaises, entregar.

CorreioTests.test_aceite_smtp_nao_significa_entrega

Linhas 13 a 18. Assinatura: test_aceite_smtp_nao_significa_entrega(self).

Cenário automatizado: aceite smtp nao significa entrega. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: patch.dict, patch, self.assertEqual, canal.starttls.assert_called_once, canal.login.assert_called_once, entregar.

backend/test_datajud.py

Contratos e limites da fonte: testes locais, sem chamadas ao CNJ.

resposta

Linhas 16 a 18. Assinatura: resposta(payload, status=200, headers=None).

Auxiliar local do fluxo de test_datajud. Sua assinatura e as chamadas abaixo indicam os dados necessários; leia o corpo no ponto indicado e relacione-o ao método externo que o chama.

Chamadas para acompanhar no código: requests.Response, r.headers.update, json.dumps.encode, json.dumps.

hit

Linhas 21 a 23. Assinatura: hit(i=1, n=None).

Auxiliar local do fluxo de test_datajud. Sua assinatura e as chamadas abaixo indicam os dados necessários; leia o corpo no ponto indicado e relacione-o ao método externo que o chama.

Chamadas para acompanhar no código: str, numero.

pagina

Linhas 26 a 27. Assinatura: pagina(itens, total=None, **extra).

Auxiliar local do fluxo de test_datajud. Sua assinatura e as chamadas abaixo indicam os dados necessários; leia o corpo no ponto indicado e relacione-o ao método externo que o chama.

Chamadas para acompanhar no código: len.

TransporteDataJudTests

Linha 30. Classe com base unittest.TestCase.

TransporteDataJudTests.chamar

Linhas 31 a 35. Assinatura: chamar(self, respostas).

Auxiliar local do fluxo de test_datajud. Sua assinatura e as chamadas abaixo indicam os dados necessários; leia o corpo no ponto indicado e relacione-o ao método externo que o chama.

Chamadas para acompanhar no código: Mock, patch.dict, patch, DataJud.buscar, numero, DataJud.

TransporteDataJudTests.test_consulta_pequena_com_ordenacao_publica_e_projecao

Linhas 37 a 45. Assinatura: test_consulta_pequena_com_ordenacao_publica_e_projecao(self).

Cenário automatizado: consulta pequena com ordenacao publica e projecao. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.chamar, self.assertEqual, self.assertNotIn, self.assertIn, self.assertIs, sessao.close.assert_called_once, len, resposta, pagina, hit.

TransporteDataJudTests.test_paginacao_cursor_preserva_todos_registros

Linhas 47 a 50. Assinatura: test_paginacao_cursor_preserva_todos_registros(self).

Cenário automatizado: paginacao cursor preserva todos registros. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.chamar, self.assertEqual, len, resposta, pagina, hit, range.

TransporteDataJudTests.test_pagina_cheia_com_total_exato_nao_consulta_de_novo

Linhas 52 a 54. Assinatura: test_pagina_cheia_com_total_exato_nao_consulta_de_novo(self).

Cenário automatizado: pagina cheia com total exato nao consulta de novo. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.chamar, self.assertEqual, resposta, pagina, hit, range.

TransporteDataJudTests.test_recusa_respostas_parciais_e_processo_inesperado

Linhas 56 a 63. Assinatura: test_recusa_respostas_parciais_e_processo_inesperado(self).

Cenário automatizado: recusa respostas parciais e processo inesperado. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: pagina, self.assertEqual, self.subTest, self.assertRaises, self.chamar, hit, resposta, numero.

TransporteDataJudTests.test_recusa_repeticao_de_pagina

Linhas 65 a 67. Assinatura: test_recusa_repeticao_de_pagina(self).

Cenário automatizado: recusa repeticao de pagina. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertRaises, self.chamar, resposta, pagina, hit, range.

TransporteDataJudTests.test_limite_e_sobrecarga_sao_distintos

Linhas 69 a 74. Assinatura: test_limite_e_sobrecarga_sao_distintos(self).

Cenário automatizado: limite e sobrecarga sao distintos. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, self.subTest, self.assertRaises, self.chamar, resposta.

TransporteDataJudTests.test_sobrecarga_encapsulada_como_resposta_real_do_cnj

Linhas 76 a 87. Assinatura: test_sobrecarga_encapsulada_como_resposta_real_do_cnj(self).

Cenário automatizado: sobrecarga encapsulada como resposta real do cnj. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, self.assertNotIn, DataJud.tipos_erro, set, self.subTest, self.assertRaises, self.chamar, str, resposta.

TransporteDataJudTests.test_timeout_tls_proxy_e_rede_tem_diagnosticos

Linhas 89 a 94. Assinatura: test_timeout_tls_proxy_e_rede_tem_diagnosticos(self).

Cenário automatizado: timeout tls proxy e rede tem diagnosticos. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, self.assertGreater, requests.Timeout, requests.exceptions.SSLError, requests.exceptions.ProxyError, requests.ConnectionError, self.subTest, self.assertRaises, self.chamar.

TransporteDataJudTests.test_credencial_recusada_nao_e_processo_inexistente

Linhas 96 a 99. Assinatura: test_credencial_recusada_nao_e_processo_inexistente(self).

Cenário automatizado: credencial recusada nao e processo inexistente. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, self.assertNotIn, self.assertRaises, self.chamar, str, resposta.

EsperaDataJudTests

Linha 102. Classe com base unittest.TestCase.

EsperaDataJudTests.test_espera_impede_repeticoes_e_preserva_referencia

Linhas 103 a 129. Assinatura: test_espera_impede_repeticoes_e_preserva_referencia(self).

Cenário automatizado: espera impede repeticoes e preserva referencia. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: tempfile.TemporaryDirectory, Equipes, c.usuario, Mock, c.autenticar, c.adicionar, patch.dict, patch, c.consultar, FalhaAPI, self.assertEqual, self.assertGreaterEqual, Path, c.contexto.

backend/test_equipes.py

Módulo de testes de comportamento do projeto.

numero

Linhas 14 a 16. Assinatura: numero(tribunal='817', sequencia='0000001').

Gera um número de formato CNJ com dígitos verificadores para testes de validação. É dado sintético, não prova de existência de processo.

Chamadas para acompanhar no código: str.zfill, str, int.

EquipesTests

Linha 19. Classe com base unittest.TestCase.

EquipesTests.setUp

Linhas 20 a 24. Assinatura: setUp(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: pasta.mkdir, tempfile.TemporaryDirectory, Equipes, self.usuario, Path, self.c.contexto, Path.resolve.

EquipesTests.tearDown

Linhas 26 a 26. Assinatura: tearDown(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: self.tmp.cleanup.

EquipesTests.usuario

Linhas 28 a 28. Assinatura: usuario(self, email).

Resolve um token em usuário somente quando a sessão existe e ainda é válida.

Chamadas para acompanhar no código: self.c.usuario, self.c.autenticar.

EquipesTests.test_cnj

Linhas 30 a 33. Assinatura: test_cnj(self).

Cenário automatizado: cnj. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, validar_numero, numero, self.assertRaises.

EquipesTests.test_migracao_preserva_conta_e_caso

Linhas 35 a 39. Assinatura: test_migracao_preserva_conta_e_caso(self).

Cenário automatizado: migracao preserva conta e caso. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: Caderno, antigo.usuario, Equipes, self.assertEqual, antigo.autenticar, antigo.adicionar, Path, numero, novo2.listar.

EquipesTests.test_convite_isolamento_e_revogacao

Linhas 41 a 49. Assinatura: test_convite_isolamento_e_revogacao(self).

Cenário automatizado: convite isolamento e revogacao. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.usuario, self.c.aceitar, self.assertEqual, self.c.revogar, self.c.demonstracao, self.assertRaises, self.c.detalhe, self.c.convidar, self.c.listar.

EquipesTests.test_conferencia_por_versao_e_notificacao_unica

Linhas 51 a 60. Assinatura: test_conferencia_por_versao_e_notificacao_unica(self).

Cenário automatizado: conferencia por versao e notificacao unica. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertNotEqual, self.c.revisar, self.assertEqual, self.c.consultar, self.c.demonstracao, self.c.conexao, db.execute, self.c.listar, db.execute.fetchone.

EquipesTests.test_falha_preserva_referencia

Linhas 62 a 67. Assinatura: test_falha_preserva_referencia(self).

Cenário automatizado: falha preserva referencia. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.consultar, Mock, FalhaAPI, self.assertEqual, self.c.demonstracao, self.assertRaises, self.c.listar, self.c.historico.

EquipesTests.test_concorrencia_reserva_antes_de_consultar

Linhas 69 a 81. Assinatura: test_concorrencia_reserva_antes_de_consultar(self).

Cenário automatizado: concorrencia reserva antes de consultar. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: threading.Event, threading.Thread, th.start, self.assertTrue, self.assertEqual, self.c.demonstracao, entrou.wait, liberar.set, th.join, entrou.set, liberar.wait, Demonstracao.buscar, self.c.consultar, self.assertRaises.

EquipesTests.test_concorrencia_reserva_antes_de_consultar.Lenta

Linha 71. Classe com base object.

EquipesTests.test_concorrencia_reserva_antes_de_consultar.Lenta.buscar

Linhas 72 a 72. Assinatura: buscar(s, t, n).

Obtém dados segundo o contrato da fonte. DataJud usa HTTP JSON; DataJudValidado acrescenta validação; Demonstracao retorna conteúdo fictício controlado.

Chamadas para acompanhar no código: entrou.set, liberar.wait, Demonstracao.buscar, Demonstracao.

EquipesTests.test_concorrencia_reserva_antes_de_consultar.executar

Linhas 73 a 75. Assinatura: executar().

Executa uma operação no contexto do chamador. No worker, chama a consulta elegível; no roteiro Playwright, conduz o percurso de navegador.

Chamadas para acompanhar no código: self.c.consultar, Lenta, erros.append.

EquipesTests.test_importacao_e_duplicacao

Linhas 83 a 88. Assinatura: test_importacao_e_duplicacao(self).

Cenário automatizado: importacao e duplicacao. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.importar, self.assertEqual, self.c.listar, len, numero.

EquipesTests.test_agendamento_e_backup_restaurado

Linhas 90 a 96. Assinatura: test_agendamento_e_backup_restaurado(self).

Cenário automatizado: agendamento e backup restaurado. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.alterar, self.c.tick, self.assertTrue, copiar, self.assertEqual, self.c.demonstracao, self.c.conexao, db.execute, Path, self.assertRaises, self.c.listar, Equipes.listar, Equipes.

EquipesTests.test_email_local_sem_envio_externo

Linhas 98 a 104. Assinatura: test_email_local_sem_envio_externo(self).

Cenário automatizado: email local sem envio externo. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.consultar, self.c.email_tick, self.assertEqual, self.c.demonstracao, self.c.conexao, db.execute, len, list, BinOp.glob, Path.

backend/test_fila_datajud.py

Fila persistente e recuperação da fonte, sem chamadas externas.

FilaDataJudTests

Linha 14. Classe com base unittest.TestCase.

FilaDataJudTests.setUp

Linhas 15 a 23. Assinatura: setUp(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: tempfile.TemporaryDirectory, Piloto, self.c.usuario, patch.dict, self.env.start, Path, self.c.autenticar, self.c.conexao, db.execute, self.c.adicionar, self.c.contexto, numero.

FilaDataJudTests.tearDown

Linhas 25 a 25. Assinatura: tearDown(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: self.env.stop, self.tmp.cleanup.

FilaDataJudTests.liberar

Linhas 27 a 31. Assinatura: liberar(self).

Libera uma sincronização do cenário concorrente de teste para permitir que a consulta simulada prossiga.

Chamadas para acompanhar no código: self.c.conexao, db.execute.

FilaDataJudTests.test_fila_idempotente_persistida_espera_e_recupera_sem_perder_versao

Linhas 33 a 58. Assinatura: test_fila_idempotente_persistida_espera_e_recupera_sem_perder_versao(self).

Cenário automatizado: fila idempotente persistida espera e recupera sem perder versao. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: Mock, self.c.consultar, self.liberar, patch, self.c.agendar_consulta, self.assertEqual, Piloto, FalhaAPI, self.c.consultas_tick, self.assertGreater, self.assertIsNone, self.c.conexao, numero, self.c.carteira.

FilaDataJudTests.test_para_em_cinco_falhas_sem_loop_e_nao_repete_credencial

Linhas 60 a 71. Assinatura: test_para_em_cinco_falhas_sem_loop_e_nao_repete_credencial(self).

Cenário automatizado: para em cinco falhas sem loop e nao repete credencial. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: Mock, FalhaAPI, patch, self.c.agendar_consulta, range, self.liberar, self.c.consultas_tick, self.assertEqual, self.c.carteira.

FilaDataJudTests.test_autorizacao_email_e_arquivamento_protegem_fila

Linhas 73 a 83. Assinatura: test_autorizacao_email_e_arquivamento_protegem_fila(self).

Cenário automatizado: autorizacao email e arquivamento protegem fila. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.usuario, self.assertEqual, self.c.autenticar, self.assertRaises, self.c.agendar_consulta, self.c.conexao, db.execute.

FilaDataJudTests.test_fonte_desativada_pausa_fila_sem_busca_externa

Linhas 85 a 91. Assinatura: test_fonte_desativada_pausa_fila_sem_busca_externa(self).

Cenário automatizado: fonte desativada pausa fila sem busca externa. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.agendar_consulta, patch.dict, patch, self.c.consultas_tick, fonte.assert_not_called, self.c.conexao, db.execute.fetchone, self.assertEqual, db.execute.

backend/test_piloto.py

Regressões de piloto com SQLite real e tokens locais sem envio externo.

PilotoTests

Linha 15. Classe com base unittest.TestCase.

PilotoTests.setUp

Linhas 16 a 21. Assinatura: setUp(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: tempfile.TemporaryDirectory, Piloto, self.usuario, Path, self.c.contexto.

PilotoTests.tearDown

Linhas 23 a 23. Assinatura: tearDown(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: self.tmp.cleanup.

PilotoTests.usuario

Linhas 25 a 26. Assinatura: usuario(self, email).

Resolve um token em usuário somente quando a sessão existe e ainda é válida.

Chamadas para acompanhar no código: self.c.usuario, self.c.autenticar.

PilotoTests.confirmar

Linhas 28 a 36. Assinatura: confirmar(self, u).

Auxiliar de teste que solicita a confirmação, lê a caixa local temporária e consome o token da conta de teste.

Chamadas para acompanhar no código: set, next, self.c.confirmar_email, BinOp.glob, patch.dict, self.c.solicitar_verificacao, self.c.conta, json.loads, p.read_text, Path.

PilotoTests.test_token_expiracao_uso_unico_e_destinatario

Linhas 38 a 50. Assinatura: test_token_expiracao_uso_unico_e_destinatario(self).

Cenário automatizado: token expiracao uso unico e destinatario. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.usuario, next, self.confirmar, self.assertTrue, patch.dict, self.c.solicitar_verificacao, BinOp.glob, json.loads, self.c.conexao, self.assertNotEqual, self.assertRaises, self.c.confirmar_email, db.execute, arquivo.read_text.

PilotoTests.test_compartilhar_exige_email_de_ambos

Linhas 52 a 61. Assinatura: test_compartilhar_exige_email_de_ambos(self).

Cenário automatizado: compartilhar exige email de ambos. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.usuario, self.confirmar, self.c.aceitar, self.c.revogar, self.assertRaises, self.c.convidar, self.c.demonstracao, self.c.detalhe.

PilotoTests.test_migracao_preserva_ids_historico_e_permite_orgs_distintas

Linhas 63 a 77. Assinatura: test_migracao_preserva_ids_historico_e_permite_orgs_distintas(self).

Cenário automatizado: migracao preserva ids historico e permite orgs distintas. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: Equipes, antigo.usuario, Piloto, self.assertEqual, novo.adicionar, antigo.autenticar, antigo.demonstracao, antigo.consultar, novo.conexao, db.execute, self.assertRaises, len, Path, antigo.contexto.

PilotoTests.test_paginacao_busca_mais_de_mil_e_isolamento

Linhas 79 a 91. Assinatura: test_paginacao_busca_mais_de_mil_e_isolamento(self).

Cenário automatizado: paginacao busca mais de mil e isolamento. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.carteira, self.assertEqual, self.usuario, self.c.conexao, db.executemany, len, self.assertRaises, self.c.exportar_escritorio, str, range.

PilotoTests.test_exportacao_sem_segredos_e_solicitacao_nao_apaga

Linhas 93 a 100. Assinatura: test_exportacao_sem_segredos_e_solicitacao_nao_apaga(self).

Cenário automatizado: exportacao sem segredos e solicitacao nao apaga. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.demonstracao, self.c.solicitar_dados, self.c.exportar_escritorio, self.assertEqual, len, self.assertNotIn.

PilotoTests.test_data_opcional_nao_vence_e_conclusao_preserva_evidencia

Linhas 102 a 111. Assinatura: test_data_opcional_nao_vence_e_conclusao_preserva_evidencia(self).

Cenário automatizado: data opcional nao vence e conclusao preserva evidencia. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.assertEqual, self.c.alterar_providencia, self.c.demonstracao, self.c.criar_providencia, self.assertRaises, self.c.rotina, self.c.resumo_diario.

backend/test_rotina.py

Módulo de testes de comportamento do projeto.

RotinaTests

Linha 10. Classe com base unittest.TestCase.

RotinaTests.setUp

Linhas 11 a 16. Assinatura: setUp(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: pasta.mkdir, tempfile.TemporaryDirectory, Escritorio, self.conta, self.c.demonstracao, self.c.consultar, Path, self.c.contexto, Path.resolve.

RotinaTests.tearDown

Linhas 18 a 18. Assinatura: tearDown(self).

Prepara ou encerra os recursos temporários do teste. Observe a criação do banco, o servidor local e o fechamento para não contaminar execuções.

Chamadas para acompanhar no código: self.tmp.cleanup.

RotinaTests.conta

Linhas 20 a 20. Assinatura: conta(self, email).

Fornece ID, e-mail e instante de verificação da própria conta. Não inclui dados de sessão ou senha.

Chamadas para acompanhar no código: self.c.usuario, self.c.autenticar.

RotinaTests.membro

Linhas 21 a 22. Assinatura: membro(self, email).

Auxiliar de teste que cria uma conta e associa o participante ao escritório do cenário.

Chamadas para acompanhar no código: self.conta, self.c.aceitar, self.c.convidar.

RotinaTests.criar

Linhas 23 a 26. Assinatura: criar(self, responsavel=None, revisor=None).

Auxiliar de teste que cria uma providência com participantes e dados controlados para validar suas transições.

Chamadas para acompanhar no código: self.c.criar_providencia, date.today.isoformat, date.today.

RotinaTests.test_fluxo_responsavel_execucao_revisor

Linhas 28 a 40. Assinatura: test_fluxo_responsavel_execucao_revisor(self).

Cenário automatizado: fluxo responsavel execucao revisor. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.membro, self.criar, self.c.alterar_providencia, self.assertEqual, self.assertRaises, len, self.c.rotina, self.c.historico_providencia.

RotinaTests.test_isolamento_e_concorrencia

Linhas 42 a 47. Assinatura: test_isolamento_e_concorrencia(self).

Cenário automatizado: isolamento e concorrencia. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.criar, self.conta, self.c.alterar_providencia, self.assertEqual, self.assertRaises, self.c.historico_providencia.

RotinaTests.test_email_envia_fora_da_transacao

Linhas 49 a 58. Assinatura: test_email_envia_fora_da_transacao(self).

Cenário automatizado: email envia fora da transacao. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.c.consultar, self.c.conexao, db.execute, patch, self.c.email_tick, envio.assert_called_once, self.assertEqual, db.execute.fetchone.

RotinaTests.test_email_envia_fora_da_transacao.rede

Linhas 53 a 56. Assinatura: rede(*args, **kwargs).

Simulação do canal externo de e-mail no teste. Serve para verificar a interação com o banco durante o envio sem transmissão externa.

Chamadas para acompanhar no código: self.c.conexao, db.execute.

RotinaTests.test_email_revogacao_e_preferencia

Linhas 60 a 70. Assinatura: test_email_revogacao_e_preferencia(self).

Cenário automatizado: email revogacao e preferencia. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.membro, self.criar, self.c.revogar, self.assertEqual, self.c.conexao, db.execute, patch, self.c.email_tick, envio.assert_not_called, self.c.rotina, db.execute.fetchone.

RotinaTests.test_reprocessamento_exige_admin

Linhas 72 a 78. Assinatura: test_reprocessamento_exige_admin(self).

Cenário automatizado: reprocessamento exige admin. Leia os assert para identificar o resultado exigido e os mocks para distinguir dependências simuladas.

Chamadas para acompanhar no código: self.membro, self.c.alterar, self.assertEqual, self.c.reprocessar, self.c.conexao, db.execute, self.assertRaises, self.c.operacao.

frontend/acesso.js

JavaScript executado no navegador. Eventos de formulário e clique também aparecem no nível superior do arquivo; não são métodos Python.

showMode

Linha 6. Parâmetros: next.

Alterna cadastro e entrada e ajusta descrições, senha e ação do formulário.

Eventos e formulários vinculados: access-register, access-login, access-form.

frontend/acompanhamento.js

JavaScript executado no navegador. Eventos de formulário e clique também aparecem no nível superior do arquivo; não são métodos Python.

acompanharAtualizacoes

Linha 18. Parâmetros: nenhum.

Agenda leitura do estado local da fila, sem consultar o CNJ. Evita atualização durante edição, histórico aberto, aba oculta ou outra área; encerra quando não há pedido pendente.

status

Linha 28. Parâmetros: message, kind = 'success'.

Mostra mensagem de andamento, sucesso ou erro na região anunciada da página.

signedIn

Linha 34. Parâmetros: value.

Alterna áreas de entrada e trabalho. A autorização real continua na API.

api

Linha 43. Parâmetros: path, data.

Centraliza fetch, JSON, timeout e erros; identifica sessão expirada e registra caminhos bem-sucedidos para o guia.

run

Linha 66. Parâmetros: action, loading = 'Processando sua solicitação…'.

Impede ações simultâneas, marca áreas como ocupadas e garante liberação e foco ao terminar, inclusive em erro.

button

Linha 88. Parâmetros: label, action, primary = false.

Cria botão com estilo compartilhado e encaminha a ação pelo controle run.

field

Linha 95. Parâmetros: form, label, value = '', type = 'input'.

Cria campo e rótulo em um formulário para manter estrutura consistente.

load

Linha 110. Parâmetros: nenhum.

Carrega contexto, carteira paginada, membros, avisos e painéis do escritório selecionado.

renderTeam

Linha 139. Parâmetros: admin.

Apresenta membros e ações de revogação conforme o papel recebido; a API ainda verifica permissão.

renderMetrics

Linha 156. Parâmetros: nenhum.

Mostra indicadores calculados no banco inteiro e orienta o percurso de demonstração.

render

Linha 179. Parâmetros: nenhum.

Atualiza a lista e seus controles. No laboratório usa estado local; na carteira apresenta a página obtida do servidor.

caseCard

Linha 198. Parâmetros: c.

Constrói a representação de um processo com fonte, referência, falha e ações disponíveis.

review

Linha 263. Parâmetros: id, version.

Registra leitura de uma versão concreta e atualiza os dados apresentados.

editor

Linha 268. Parâmetros: c.

Cria os controles de organização e arquivamento de um processo e envia alterações à API.

history

Linha 298. Parâmetros: id, name.

Carrega versões e tentativas, apresenta diferenças e oferece registro de leitura e providência.

submit

Linha 337. Parâmetros: id, action.

Converte os dados de um formulário em objeto e executa sua ação com controle de carregamento.

iniciarAplicacao

Linha 421. Parâmetros: nenhum.

Inicializa a área de trabalho apenas uma vez depois que as dependências da página estão prontas.

el

Linha 3. Função de expressão com parâmetros id.

Localiza um elemento por ID para os componentes da área persistente.

text

Linha 4. Função de expressão com parâmetros tag, value, className.

Cria um elemento e escreve texto com textContent, sem interpretar conteúdo recebido como HTML.

date

Linha 104. Função de expressão com parâmetros value.

Formata instantes Unix em segundos para leitura local. Um valor ausente é apresentado como ainda não consultado.

movementDate

Linha 105. Função de expressão com parâmetros value.

Tenta formatar a data recebida da movimentação e conserva o valor original se não puder interpretá-lo.

Eventos e formulários vinculados: auth-form, case-form, recover-form, reset-form, office-form, invite-form, accept-form, close-history, logout, demo, organization, search, filter, csv-input.

frontend/amber.js

JavaScript executado no navegador. Eventos de formulário e clique também aparecem no nível superior do arquivo; não são métodos Python.

escapeHTML

Linha 35. Parâmetros: value.

Escapa caracteres especiais antes de montar trechos controlados de HTML do laboratório.

notify

Linha 36. Parâmetros: message.

Apresenta aviso temporário no treino e cancela o temporizador anterior.

showPage

Linha 37. Parâmetros: page.

Alterna as seções do laboratório e seus controles de navegação.

visibleCases

Linha 38. Parâmetros: nenhum.

Aplica busca e filtro sobre o pequeno catálogo fictício local.

statusOf

Linha 39. Parâmetros: c.

Produz o rótulo do estado do caso de treino, preservando a distinção entre falha, incompletude e conferência.

render

Linha 40. Parâmetros: nenhum.

Atualiza a lista e seus controles. No laboratório usa estado local; na carteira apresenta a página obtida do servidor.

renderCatalog

Linha 46. Parâmetros: nenhum.

Apresenta cenários fictícios e a opção de adicioná-los ao treino.

log

Linha 47. Parâmetros: title,description.

Acrescenta ação ao histórico da sessão do laboratório.

renderHistory

Linha 48. Parâmetros: nenhum.

Desenha eventos do treino; esse histórico local não equivale às tabelas persistentes da equipe.

moveCoachToBody

Linha 49. Parâmetros: nenhum.

Retira o guia de dentro de um diálogo antes de fechá-lo, evitando ocultação acidental.

closeDialog

Linha 50. Parâmetros: id.

Fecha o diálogo e coordena os eventos de avanço do tutorial.

openDetail

Linha 51. Parâmetros: id.

Apresenta comparação antes/depois de um caso do laboratório.

carregarMovimentacoes

Linha 52. Parâmetros: targets.

Auxiliar do módulo amber. Siga as entradas e as alterações de estado no corpo antes de modificar sua chamada.

consult

Linha 70. Parâmetros: nenhum.

Auxiliar do módulo amber. Siga as entradas e as alterações de estado no corpo antes de modificar sua chamada.

exportCSV

Linha 79. Parâmetros: nenhum.

Auxiliar do módulo amber. Siga as entradas e as alterações de estado no corpo antes de modificar sua chamada.

clearHighlight

Linha 80. Parâmetros: nenhum.

Auxiliar do módulo amber. Siga as entradas e as alterações de estado no corpo antes de modificar sua chamada.

updateCoach

Linha 81. Parâmetros: nenhum.

Reposiciona e atualiza o guia contextual do laboratório.

advance

Linha 86. Parâmetros: event,id.

Avança uma etapa de treino quando ocorre o evento esperado.

carregarFonteTreino

Linha 97. Parâmetros: nenhum.

Carrega portal.html, verifica HTTP e extrai somente campos conhecidos. Mostra falha recuperável e ignora respostas antigas.

configurarCasoGuiado

Linha 143. Parâmetros: nenhum.

Adapta as instruções ao cenário fictício escolhido, em vez de fixar apenas o caso cível.

$

Linha 2. Função de expressão com parâmetros id.

Auxiliar de transformação ou seleção do módulo. A expressão devolve seu valor ao chamador; siga o corpo no ponto indicado.

frontend/guia.js

JavaScript executado no navegador. Eventos de formulário e clique também aparecem no nível superior do arquivo; não são métodos Python.

salvarGuia

Linha 28. Parâmetros: nenhum.

Persiste somente passo e estado de atividade do tutorial no navegador.

limparDestaqueGuia

Linha 29. Parâmetros: nenhum.

Remove indicações temporárias para não deixar controles destacados depois da orientação.

pausarGuia

Linha 30. Parâmetros: nenhum.

Oculta a orientação e conserva ponto de retomada; não encerra sessão nem apaga processos.

mostrarGuia

Linha 31. Parâmetros: nenhum.

Escolhe instrução e alvo da etapa atual e atualiza os controles do guia.

avancarGuia

Linha 47. Parâmetros: nenhum.

Avança a orientação ou conclui o percurso removendo os destaques.

iniciarGuia

Linha 48. Parâmetros: recomecar=false.

Inicia, retoma ou reinicia o tutorial autenticado conforme o estado da página.

demoGuia

Linha 4. Função de expressão com parâmetros nenhum.

Encontra o exemplo fictício não arquivado na página atual para orientar o percurso autenticado.

botaoCaso

Linha 5. Função de expressão com parâmetros nome.

Encontra a ação do caso fictício pelo rótulo para destacar o controle correto no guia.

temNovidade

Linha 9. Função de expressão com parâmetros c.

Verifica no snapshot fictício a presença do movimento de demonstração que representa a etapa nova.

Eventos e formulários vinculados: start-app-guide, pause-app-guide, app-guide-next, app-guide-back, app-guide-restart.

frontend/piloto.js

JavaScript executado no navegador. Eventos de formulário e clique também aparecem no nível superior do arquivo; não são métodos Python.

carregarPainelPiloto

Linha 58. Parâmetros: contexto.

Atualiza estado da conta, resumo diário e visibilidade administrativa no escritório atual.

frontend/rotina.js

JavaScript executado no navegador. Eventos de formulário e clique também aparecem no nível superior do arquivo; não são métodos Python.

atualizarRotinaContexto

Linha 16. Parâmetros: contexto.

Aplica papel e limpa referências da rotina quando a pessoa muda de escritório.

trocarArea

Linha 30. Parâmetros: rotina.

Alterna as áreas Processos e Minha rotina e os estados visuais dos controles.

abrirProvidencia

Linha 39. Parâmetros: caso, versao, nome.

Preenche formulário com processo, versão opcional, responsáveis e contexto da ação.

carregarRotina

Linha 62. Parâmetros: nenhum.

Consulta página/filtro de providências e atualiza resumo, lista e paginação.

providenciaCard

Linha 77. Parâmetros: p.

Cria a representação de uma atividade, transições permitidas, evidência e histórico.

carregarOperacao

Linha 112. Parâmetros: nenhum.

Consulta diagnóstico administrativo e disponibiliza reprocessamento autorizado das filas.

Eventos e formulários vinculados: action-form, show-routine, show-cases, refresh-routine, routine-filter, search-routine, routine-search, close-action, refresh-operation.

frontend/runtime-config.js

JavaScript executado no navegador. Eventos de formulário e clique também aparecem no nível superior do arquivo; não são métodos Python.

Este arquivo configura valores ou utiliza callbacks/funções de expressão. Leia o módulo juntamente com o capítulo correspondente do manual.

Arquivos de estrutura configuração recursos e documentação

frontend/index.html

Apresentação pública e entrada para treino e acesso.

frontend/acesso.html

Estrutura do formulário de cadastro/login e orientação dos campos.

frontend/laboratorio.html

Estrutura das telas, diálogos e controles do treino fictício.

frontend/portal.html

Fonte HTML fictícia lida pelo laboratório; não é um portal judicial.

frontend/acompanhamento.html

Estrutura da aplicação autenticada de processos, providências e equipe.

frontend/amber.css

Apresentação do laboratório, estados e componentes históricos.

frontend/acompanhamento.css

Estilo da área persistente, campos, painéis, botões e responsividade.

frontend/percurso.css

Coerência visual de apresentação/acesso/treino e ajustes de diálogos e guia.

frontend/runtime-config.js

Configuração estática de disponibilidade da API. O servidor atual fornece versão dinâmica.

frontend/assets/office.jpg

Imagem de ambiente usada na apresentação, sem conteúdo de processo.

frontend/assets/workspace.jpg

Imagem ilustrativa de espaço de trabalho.

frontend/assets/marca.svg

Recurso vetorial histórico da marca; o cabeçalho atual utiliza composição tipográfica.

package.json

Comando de testes JavaScript e dependências de desenvolvimento.

package-lock.json

Resolução exata das dependências JavaScript.

requirements.txt

Dependências Python fixadas para reproduzir o ambiente da aplicação.

Dockerfile

Receita de contêiner Linux que ainda exige validação no destino, volume persistente e HTTPS externo.

.dockerignore

Exclusões do contexto de construção para evitar incluir arquivos desnecessários.

.env.example

Exemplo das variáveis de configuração; não contém credenciais nem é carregado automaticamente.

.gitignore

Exclusões de arquivos gerados e sensíveis no versionamento.

vercel.json

Configuração da publicação estática; não hospeda automaticamente esta API Python persistente.

INICIAR.cmd

Lançador local Windows que prepara .venv, instala dependências e inicia FastAPI.

ferramentas/gerar_manual.py

Ferramenta de documentação que produz este catálogo, HTML e PDF sem ler dados privados de output.

.vscode/settings.json

Oculta dependências e temporários no explorador do VS Code para facilitar a leitura. Não apaga esses arquivos.

documentacao/causa-confirmada-datajud.md

Documento de referência: Causa confirmada da recusa DataJud. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/demonstracao-no-computador.md

Documento de referência: Demonstração no computador - 04/11/2026. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/diagnostico-e-correcao-datajud.md

Documento de referência: Consulta real do DataJud: diagnóstico e correções. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/estado-atual-do-projeto.md

Documento de referência: Entregável local do Caderno Âmbar. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/estudo-do-laboratorio.md

Documento de referência: Caderno Âmbar: aprenda usando as três bibliotecas. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/fila-atualizacoes-datajud.md

Documento de referência: Atualização do DataJud sem interromper o trabalho. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/fluxo-das-telas.md

Documento de referência: Percurso do usuário - Caderno Âmbar. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/guia-de-equipes.md

Documento de referência: Caderno Âmbar - guia da versão de equipes. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/guia-de-providencias.md

Documento de referência: Minha rotina - evolução do Caderno Âmbar. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/historico-do-laboratorio.md

Documento de referência: Caderno Âmbar. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/primeiro-guia-de-estudo.md

Documento de referência: Caderno Âmbar - código, prática e apresentação. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/prompt-correcao-datajud.md

Documento de referência: Prompt de diagnóstico e correção da consulta real. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/README.md

Documento de referência: Documentação do Caderno Âmbar. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/revisao-da-interface.md

Documento de referência: Revisão de experiência - 03/11/2026. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/teste-real-datajud.md

Documento de referência: Teste real do DataJud em 04/11/2026. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

documentacao/testes-e-validacao.md

Documento de referência: Relatório de validação - versão de equipes. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

README.md

Documento de referência: Caderno Âmbar. Consulte a data e o escopo; registros antigos não substituem o estado atual em documentacao/estado-atual-do-projeto.md.

tests/percurso.test.cjs

Testes JavaScript com jsdom. Cenários declarados: apresentação separa treino, acesso e proposta do produto; acesso mostra um modo por vez e explica os campos sem salvar senha. Respostas de rede são simuladas; não é inspeção visual.

tests/treino.test.cjs

Testes JavaScript com jsdom. Cenários declarados: treino guiado do exemplo ; fonte com falha mostra recuperação e permite tentar novamente. Respostas de rede são simuladas; não é inspeção visual.

tests/ui.test.cjs

Testes JavaScript com jsdom. Cenários declarados: fluxo de escritório, referência, novidade, histórico e conferência; primeira consulta com falha atualiza cartão e permite ler o histórico; fonte real usa fila sem bloquear dados nem repetir consulta externa; sobrecarga confirmada do fornecedor tem mensagem específica sem expor diagnóstico bruto; troca de escritório limpa filtros e respeita o papel de membro; painel da conta orienta confirmação sem expor código na resposta; importação exige prévia e invalida aprovação quando CSV muda; marca sem ícone, recursos versionados e ações agrupadas; processo gera providência e a rotina acompanha execução e revisão; guia acompanha ações bem-sucedidas e pode

pausar e retomar. Respostas de rede são simuladas; não é inspeção visual.

Diretórios que não são fonte para estudar linha por linha

.git guarda metadados de versionamento. node_modules e .venv são dependências instaladas. __pycache__ guarda bytecode gerado. output guarda dados privados e artefatos de execução. Este gerador não copia o conteúdo desses diretórios para o manual. O banco, backups e mensagens devem ser protegidos, não anexados a uma apresentação pública.

Entregável local do Caderno Âmbar

Revisão de 4 de novembro de 2026. Consulta autenticada real do TJPE validada com 35 movimentos, versão, histórico e autoria. Veja diagnostico-e-correcao-datajud.md. Este arquivo é o estado atual; os guias V2 e de rotina preservam a explicação das etapas anteriores.

Implementado nesta revisão

• Fila persistente DataJud: solicitação HTTP 202, processamento em segundo plano, espera crescente, até cinco falhas, pausa de erros de configuração e acompanhamento na interface. Os últimos dados válidos permanecem acessíveis. Ver `fila-atualizacoes-datajud.md`.

• Serviço `Piloto`, sobre a rotina existente, com backup anterior à reconstrução da tabela de processos.

• Restrição de duplicação por escritório, número e tribunal, preservando IDs e vínculos. A migração falha com rollback se houver conflitos ou vínculos inválidos.

• `GET /api/carteira` com busca, filtros, contagem e paginação de 20 itens no banco. A interface deixou de depender da rota legada limitada a 1.000 registros.

• Indicadores de toda a carteira e resumo das providências do usuário, vencidas e para revisão.

• Confirmação de e-mail com token aleatório, hash no banco, expiração de 30 minutos, uso único e limites de tentativas. O código não é devolvido na API.

• Confirmação exigida para criar e aceitar convites e consultar fontes reais. Demonstrações continuam disponíveis para estudo. Contas antigas não são consideradas verificadas automaticamente.

• Exportação administrativa JSON do escritório, com versões, consultas, providências e eventos. Senhas, sessões, convites e tokens não são exportados. O CSV permanece disponível.

• Registro administrativo de solicitações de retenção/exclusão. Não equivale a execução de exclusão nem a atendimento automático de uma solicitação.

• Data interna da providência opcional; ausência de data não conta como atraso. Conclusão não aceita apagar a evidência simultaneamente.

• Produção recusa inicialização sem origem HTTPS, banco configurado e SMTP. HSTS é enviado em produção. `AMBAR_WORKER=0` desativa ciclos em instâncias HTTP adicionais.

Uso local

Execute `INICIAR.cmd` em um computador com Python instalado ou use um ambiente virtual que contenha `requirements.txt`. Abra a apresentação na porta 8093. A rota recomendada é `backend/aplicacao.py`; o servidor legado 8092 não atende ao contrato atual completo.

Na área de trabalho, expanda Minha conta e confirmação de e-mail. Solicite o código. Em desenvolvimento, abra o JSON correspondente ao seu e-mail em `output/email-local`, copie apenas

`token` e confirme. Isso é simulação local de entrega; não comprova propriedade de uma caixa externa. Em produção, o canal SMTP envia a mensagem ao destinatário.

Pendências que impedem declarar produção concluída

Fonte real autorizada e carteira consentida ainda precisam de validação externa. SMTP real, confirmação de entrega pelo provedor, hospedagem, HTTPS público e backup externo protegido não foram configurados nem testados. MFA de administrador não foi implementado. A política de retenção e a exclusão efetiva de dados e backups dependem de definição e implementação; há apenas registro de solicitação. Não existe integração de intimações oficiais, cálculo de prazo judicial, peticionamento, financeiro ou armazenamento de anexos privados.

O worker ainda roda no processo da aplicação. A flag de configuração não impede que um operador inicie dois workers por engano; as reservas existentes reduzem duplicação por tarefa, mas isso não equivale a liderança distribuída. A implantação deve ter uma única instância de ciclos, supervisão e monitoramento. A rota de carteira está tipada na entrada; as rotas legadas continuam em um roteador genérico e precisam de contratos OpenAPI individuais.

As exportações administrativas contêm dados do escritório: guarde-as em armazenamento protegido e controle o acesso. Não substituem o backup consistente do banco. A exportação não é ainda um importador de restauração.

Escopo de validação

Os resultados atuais devem ser lidos junto de `testes-e-validacao.md`: 56 testes Python e 16 de interface aprovados na revalidação de 05/11/2026. Os testes usam SQLite temporário, HTTP local e DOM simulado. A demonstração adicional no navegador confirmou cartões, providência, solicitação em segundo plano e laboratório fictício. Isso não substitui validação completa de acessibilidade e dispositivos.

Sequência de implantação

1. Aprovar finalidade, fonte, acesso e carteira de piloto com o escritório. 2. Definir responsável operacional, política de retenção, MFA e tratamento de incidentes. 3. Configurar domínio HTTPS, API persistente, SMTP e armazenamento de backup externo protegido. Usar segredos do serviço, sem gravá-los no Git. 4. Executar uma única instância de agendamento; instâncias HTTP adicionais usam `AMBAR_WORKER=0`. Fazer build e testar Docker no ambiente de destino antes de adotá-lo. 5. Restaurar uma cópia em ambiente isolado e percorrer o fluxo completo com duas contas e dois escritórios. 6. Testar desktop, celular, teclado, falhas de rede e expiração de sessão em navegador permitido. 7. Fazer piloto acompanhado e medir falhas, atrasos da fonte, tempo para conferir e conclusão das providências. Só ampliar após corrigir problemas observados.